

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN-ĐIỆN TỬ



HCM-UTE

ĐỒ ÁN 1

**Đề tài: Hệ thống quản lý vườn thông minh ứng dụng
IoT**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

GVHD: ThS. Vũ Chí Cường

Sinh viên thực hiện:

Nguyễn Phi Hùng 23119152

Võ Đăng Khoa 23119153

TP. HỒ CHÍ MINH - NĂM 2026

LỜI CẢM ƠN

Nhóm xin gửi lời cảm ơn chân thành đến thầy Vũ Chí Cường đã tận tình định hướng, góp ý và đồng hành cùng nhóm trong suốt quá trình thực hiện đề tài. Những ý kiến phản hồi của thầy không chỉ giúp nhóm giải quyết các vấn đề kỹ thuật mà còn là động lực lớn để nhóm hoàn thiện sản phẩm đến mức tốt nhất có thể.

Nhóm cũng xin trân trọng cảm ơn quý thầy cô Khoa Điện - Điện tử đã truyền đạt kiến thức nền tảng trong suốt những năm học vừa qua. Những kiến thức về vi điều khiển, mạch điện tử, lập trình nhúng và hệ thống IoT mà nhóm tiếp thu được trên giảng đường chính là nền tảng để triển khai đề tài này.

Cuối cùng, dù đã cố gắng hết sức, đề tài chắc chắn vẫn còn những thiếu sót. Nhóm rất mong nhận được sự góp ý của quý thầy cô để có thể hoàn thiện hơn trong tương lai.

MỤC LỤC

PHẦN I: TỔNG QUAN	1
1. Lý do chọn đề tài	1
2. Mục tiêu của đề tài	1
3. Phạm vi nghiên cứu	1
4. Phương pháp nghiên cứu	2
5. Nội dung	2
6. Cách tiếp cận	3
PHẦN II: CƠ SỞ LÝ THUYẾT	4
1. Vi điều khiển và SBC	4
2. Các giao thức truyền thông	4
3. Cảm biến actuators và nguồn	5
PHẦN III: THIẾT KẾ VÀ THI CÔNG	6
CHƯƠNG 1: PHÂN TÍCH YÊU CẦU	6
1.1. Giới thiệu	6
1.2. Mô tả hệ thống	6
1.3. Nguyên tắc làm việc	7
1.4. Lựa chọn giải pháp	7
CHƯƠNG 2: ỨNG DỤNG	8
2.1. Phần cứng	8
2.2. Triển khai bằng phần mềm và các công cụ	9
2.2.1. STM32CubeIDE	9
2.2.2. PlatformIO	9
2.2.3. Node.js	9
2.2.4. SQLite	9
2.2.5. Tailscale	10

2.2.6. Electron	10
2.3. Cơ chế giao tiếp	10
2.3.1. Giao thức giao tiếp giữa các thành phần	10
2.3.2. Quản lý dung lượng Database bằng cơ chế FIFO	11
2.4. Các luồng hoạt động	12
2.4.1. Thu thập và hiển thị dữ liệu cảm biến	12
2.4.2. Gửi và thực thi lệnh điều khiển	13
2.4.3. Điều khiển tự động và thủ công	14
2.3.4. Giám sát hình ảnh và điều khiển camera	15
2.3.5. Cấu hình mạng WiFi cho hệ thống	16
2.3.6. Phân giải địa chỉ Server bằng mDNS	17
2.5. Tổng hợp phương án thực hiện đề tài	17
PHẦN IV: KẾT QUẢ	18
1. Đánh giá độ chính xác cảm biến	18
1.1. Đánh giá độ chính xác của cảm biến độ ẩm đất	18
1.2. Đánh giá độ chính xác của cảm biến nhiệt độ, độ ẩm	19
1.3. Đánh giá độ chính xác của cảm biến khoảng cách	21
2. Hoạt động của hệ thống	23
2.1. Quá trình boot và chọn mạng ban đầu của hệ thống	23
2.2. Giá trị cảm biến được gửi lên server và đến GUI	25
2.3. Điều khiển các thiết bị đèn, quạt, bơm và các servo từ GUI ở chế độ thủ công	26
2.4. Điều khiển các thiết bị đèn, quạt, bơm từ GUI ở chế độ tự động	31
2.5. Quan sát cây trồng với camera feed	33
PHẦN V: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	36
1. Kết luận	36
2. Hướng phát triển	36

TÀI LIỆU THAM KHẢO.....	37
--------------------------------	-----------

DANH MỤC HÌNH ẢNH

Hình 3.1: Sơ đồ hoạt động của hệ thống	6
Hình 3.2: Sơ đồ khối hệ thống	7
Hình 3.3: Sơ đồ kết nối tổng thể phần cứng	8
Hình 3.4: Sơ đồ hoạt động quản lý dữ liệu cảm biến trên Database	11
Hình 3.5: Sơ đồ hoạt động thu thập dữ liệu cảm biến	12
Hình 3.6: Sơ đồ các task FreeRTOS và cơ chế chia sẻ tài nguyên	13
Hình 3.7: Sơ đồ hoạt động của chế độ tự động và thủ công	14
Hình 3.8: Các luồng làm việc với camera	15
Hình 3.9: Lưu đồ quy trình cấu hình WiFi	16
Hình 3.10: Cách Client lấy IP của Server qua mDNS	17
Hình 4.1: Mạng do ESP32 tạo	23
Hình 4.2: Nhập mật khẩu mạng	24
Hình 4.3: ESP32 nhận được hai phản hồi ACK từ CAM và Server	25
Hình 4.4: ESP32 tiến hành kết nối với mạng WiFi được chọn	25
Hình 4.5: ESP32 phân giải địa chỉ của Server với mDNS	25
Hình 4.6: ESP32 yêu cầu dữ liệu cảm biến và gửi lên Server	25
Hình 4.7: Các giá trị vừa được ESP32 đẩy lên Server hiển thị trên GUI	26
Hình 4.8: Dữ liệu cảm biến trong database được dùng để xây dựng biểu đồ	26
Hình 4.9: Giao diện tab điều khiển thiết bị	27
Hình 4.10: Hệ thống khi chưa bật biết bị	27
Hình 4.11: ESP32 nhận lệnh bật đèn từ Server	28
Hình 4.12: Nhấn nút Light(ON) để bật đèn	28
Hình 4.13: Đèn của hệ thống đã bật	29
Hình 4.14: Giao diện tab điều khiển Camera	29
Hình 4.15: ESP32 nhận lệnh điều khiển servo x	30

Hình 4.16: Đặt góc quay tùy chỉnh	30
Hình 4.17: ESP32 nhận lệnh điều khiển servo y	30
Hình 4.18: Thay đổi góc quay - động cơ quay theo	31
Hình 4.19: Giao diện tab điều chỉnh ngưỡng hoạt động cho chế độ tự động	32
Hình 4.20: ESP32 nhận lệnh điều khiển quạt 1	33
Hình 4.21: Hệ thống tự động bật thiết bị dựa trên thông tin môi trường đã thiết lập ngưỡng	33
Hình 4.22: Tùy chỉnh các thông số Camera	34
Hình 4.23: Camera thay đổi theo thông số đã chỉnh	35

PHẦN I: TỔNG QUAN

1. Lý do chọn đề tài

Trong bối cảnh chuyển đổi số trong nông nghiệp, nhu cầu ứng dụng công nghệ vào quản lý vườn cây nhằm nâng cao hiệu quả sản xuất và tiết kiệm tài nguyên ngày càng trở nên cấp thiết. Tuy nhiên, công tác quản lý vườn hiện nay, bao gồm tưới tiêu, điều chỉnh ánh sáng và kiểm soát các yếu tố môi trường, vẫn chủ yếu dựa vào phương pháp thủ công, thiếu tính tự động hóa và giám sát tập trung. Điều này dẫn đến tình trạng lãng phí nước, điện và nhân công, đồng thời khó đảm bảo các điều kiện sinh trưởng tối ưu cho cây trồng, làm giảm năng suất và hiệu quả kinh tế. Bên cạnh đó, việc giám sát từ xa và quản lý dữ liệu môi trường theo thời gian thực còn nhiều hạn chế, gây khó khăn cho người trồng trong việc đưa ra quyết định kịp thời.

Xuất phát từ thực trạng đó, nhóm nhận thấy việc xây dựng một hệ thống vườn thông minh ứng dụng IoT là một hướng nghiên cứu phù hợp và có tính thực tiễn cao, nhằm giải quyết các bài toán quản lý và vận hành vườn trồng một cách hiệu quả.

Đề tài còn giúp nhóm có cơ hội vận dụng kiến thức đã học về hệ thống nhúng, truyền thông IoT và phát triển ứng dụng để giải quyết một bài toán cụ thể trong lĩnh vực nông nghiệp công nghệ cao. Do đó, nhóm quyết định lựa chọn đề tài “Xây dựng hệ thống quản lý vườn thông minh” để nghiên cứu và thực hiện trong đồ án này.

2. Mục tiêu của đề tài

Nghiên cứu, thiết kế và xây dựng hệ thống quản lý vườn thông minh ứng dụng IoT, cho phép giám sát các thông số môi trường và điều khiển tự động các thiết bị trong vườn nhằm tối ưu hóa điều kiện sinh trưởng cho cây trồng, nâng cao hiệu quả sản xuất và tiết kiệm tài nguyên.

3. Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu và xây dựng hệ thống vườn thông minh ứng dụng IoT, bao gồm các nội dung chính sau:

- Nghiên cứu các kỹ thuật thu thập dữ liệu từ cảm biến môi trường như nhiệt độ, độ ẩm không khí, độ ẩm đất, mực nước và ánh sáng phục vụ cho việc giám sát và điều khiển.
- Nghiên cứu cơ chế điều khiển tự động các thiết bị đầu ra (đèn, quạt, bơm) dựa trên dữ liệu cảm biến và ngưỡng cài đặt, kết hợp với chế độ điều khiển thủ công.
- Nghiên cứu xây dựng hệ thống Server xử lý và lưu trữ dữ liệu tập trung, cung cấp các API cho phép các thiết bị và giao diện người dùng giao tiếp.
- Nghiên cứu thiết kế giao diện giám sát và điều khiển trực quan, cho phép người dùng theo dõi dữ liệu thời gian thực và thao tác từ xa.
- Nghiên cứu giải pháp kết nối an toàn cho phép truy cập hệ thống từ các mạng khác nhau.

Phạm vi nghiên cứu không bao gồm các vấn đề về mô hình canh tác, kỹ thuật trồng trọt chuyên sâu, các giống cây trồng cụ thể hay phân tích dữ liệu lớn. Hệ thống chỉ tập trung vào mô hình giám sát và điều khiển môi trường để hỗ trợ quản lý vườn trồng.

4. Phương pháp nghiên cứu

Thu thập, nghiên cứu và tổng hợp tài liệu liên quan đến IoT, hệ thống nhúng, vi điều khiển, cảm biến, truyền thông và các giải pháp kết nối mạng an toàn. Phân tích yêu cầu, xác định các thành phần chức năng, thiết kế kiến trúc tổng thể, sơ đồ kết nối và luồng dữ liệu, lựa chọn linh kiện phù hợp. Xây dựng hệ thống phần cứng, lập trình firmware cho vi điều khiển, phát triển Server và giao diện GUI, thiết lập kết nối mạng.

Kiểm thử từng module và tích hợp toàn hệ thống, đánh giá hiệu năng dựa trên các tiêu chí về năng lượng tiêu thụ, độ ổn định, độ chính xác và tính tiện dụng. Phân tích kết quả thực nghiệm, tổng hợp ưu nhược điểm và đề xuất hướng cải tiến hệ thống.

5. Nội dung

Nội dung được triển khai theo các phần chính như sau:

Phần 1: Tổng quan đề tài

Phần này trình bày lý do chọn đề tài, mục tiêu nghiên cứu, đối tượng và phạm vi nghiên cứu, đồng thời giới thiệu tổng quan về chuyển đổi số trong nông nghiệp và vai trò của IoT trong mô hình vườn thông minh.

Phần 2: Cơ sở lý thuyết

Phần này trình bày các khái niệm cơ bản có liên quan đến đề tài

Phần 3: Nội dung nghiên cứu

Đây là phần trọng tâm của đề tài, bao gồm hai chương:

Chương 1: Phân tích và thiết kế hệ thống

Trình bày việc phân tích yêu cầu, đề xuất kiến trúc tổng thể của hệ thống vườn thông minh, đồng thời thiết kế các thành phần chính như thiết bị IoT, vi điều khiển (MCU), Server, cơ sở dữ liệu và giao diện người dùng.

Chương 2: Xây dựng và triển khai hệ thống vườn thông minh.

Trình bày quá trình xây dựng và triển khai ứng dụng vườn thông minh dựa trên thiết kế đã đề xuất, mô tả nguyên lý hoạt động, luồng xử lý dữ liệu và đánh giá hiệu quả của hệ thống trong thực tế.

Phần 4: Phân tích và đánh giá kết quả

Phần này sẽ dựa trên kết quả thực nghiệm, tính toán độ chính xác của các module cụ thể để đánh giá hiệu suất và tính ổn định của hệ thống.

Phần 5: Kết luận và hướng phát triển

Phần này tổng kết các kết quả đạt được của đề tài, nêu lên những ưu điểm, hạn chế còn tồn tại, đồng thời đề xuất các hướng phát triển và mở rộng hệ thống trong tương lai.

6. Cách tiếp cận

Nhóm tiến hành nghiên cứu các công nghệ liên quan đến IoT, vi điều khiển, cảm biến và giải pháp kết nối mạng, phân tích yêu cầu và thiết kế kiến trúc tổng thể cùng sơ đồ kết nối giữa các thành phần, tiến hành lắp đặt phần cứng, lập trình firmware cho vi điều khiển, phát triển Server và giao diện GUI, thiết lập kết nối mạng và tích hợp toàn bộ thành hệ thống hoàn chỉnh. Cuối cùng, hệ thống được kiểm thử, đánh giá hiệu năng dựa trên các tiêu chí về độ ổn định, tốc độ phản hồi và khả năng phục hồi, từ đó hoàn thiện và tổng hợp kết quả báo cáo.

PHẦN II: CƠ SỞ LÝ THUYẾT

1. Vi điều khiển và SBC

STM32F103C8T6: là vi điều khiển 32-bit nhân ARM Cortex-M3 tốc độ 72MHz, Flash 64KB và RAM 20KB, hỗ trợ các ngoại vi UART, SPI, I2C, ADC, PWM. Trong hệ thống, STM32 đóng vai trò điều khiển trung tâm, thu thập dữ liệu từ cảm biến nhiệt độ, độ ẩm, ánh sáng, mực nước và điều khiển tải (đèn, quạt, bơm) qua module relay.

ESP32: là vi điều khiển tích hợp WiFi/Bluetooth kép, nhân xử lý kép Xtensa LX6 tốc độ 240MHz. Trong hệ thống, ESP32 là cầu nối truyền thông: giao tiếp với STM32 qua UART để nhận dữ liệu cảm biến và chuyển tiếp lệnh, đồng thời kết nối WiFi với Server qua WebSocket, đảm bảo phản hồi nhanh và ổn định.

ESP32-CAM: là module tích hợp ESP32 và camera OV2640 độ phân giải 2MP, truyền hình ảnh qua WiFi. Trong hệ thống, ESP32-CAM gửi luồng hình ảnh về Server để chuyển tiếp đến GUI, giúp người dùng quan sát trực quan tình trạng vườn cây từ xa.

Raspberry Pi 5: Raspberry Pi 5 là máy tính đơn bo với CPU ARM Cortex-A76 4 nhân 2,4GHz, RAM LPDDR4X lên đến 8GB. Trong hệ thống, Raspberry Pi 5 làm Server trung tâm: sử dụng Node.js host WebSocket, quản lý SQLite, thực thi logic điều khiển tự động, tiếp nhận luồng hình ảnh từ ESP32-CAM và chuyển tiếp đến GUI.

2. Các giao thức truyền thông

UART: là giao thức truyền thông nối tiếp không đồng bộ, dùng hai dây TX và RX để truyền dữ liệu giữa các thiết bị với tốc độ baud được thống nhất trước. Trong hệ thống, UART được sử dụng để STM32 và ESP32 trao đổi dữ liệu cảm biến và lệnh điều khiển trực tiếp.

I2C: là giao thức truyền thông nối tiếp đồng bộ chỉ dùng hai dây SDA và SCL, cho phép nhiều thiết bị cùng chia sẻ một bus thông qua địa chỉ định danh riêng. Đây là lựa chọn phổ biến cho các cảm biến vì tiết kiệm chân kết nối. Trong dự án, STM32 giao tiếp với cảm biến BME-280 qua I2C để đọc nhiệt độ, độ ẩm không khí và áp suất.

WebSocket: là giao thức cho phép duy trì kết nối hai chiều liên tục giữa client và server sau khi handshake qua HTTP, hỗ trợ truyền dữ liệu full-duplex theo thời gian thực. Trong hệ thống, WebSocket được dùng để ESP32 và GUI trao đổi dữ liệu với Server với tốc độ phản hồi nhanh.

mDNS: cho phép thiết bị đăng ký tên cố định (ví dụ server.local) trong mạng LAN để các thiết bị khác truy vấn lấy IP hiện tại, giải quyết vấn đề IP thay đổi khi dùng DHCP. Trong hệ thống, ESP32 sử dụng mDNS để tự động tìm địa chỉ Server thay vì code cứng IP.

Tailscale và WireGuard: WireGuard là giao thức VPN hiệu suất cao, đơn giản và độ trễ thấp. Tailscale xây dựng trên WireGuard tạo mạng riêng ảo dạng mesh, các

thiết bị kết nối trực tiếp peer-to-peer không qua trung gian. Trong hệ thống, Tailscale cho phép GUI truy cập Server từ xa an toàn mà không cần port forwarding.

SQLite: là hệ quản trị CSDL quan hệ nhưng, lưu toàn bộ dữ liệu trong một file duy nhất, không cần cài đặt hay chạy server riêng. Trong hệ thống, SQLite được Server trên Raspberry Pi sử dụng để quản lý dữ liệu cảm biến, lệnh và cấu hình thiết bị một cách nhẹ nhàng và hiệu quả.

3. Cảm biến actuators và nguồn

Cảm biến:

BME-280 đo nhiệt độ và độ ẩm không khí qua giao tiếp I2C.

Cảm biến độ ẩm đất hoạt động theo nguyên lý điện trở, đất càng ẩm điện trở càng giảm, tín hiệu analog đầu ra được đưa vào ADC của STM32 để xử lý.

Cảm biến siêu âm AJ-SR04M đo mực nước trong bồn chứa bằng cách tính thời gian phản xạ của sóng âm, được chọn vì có khả năng chống nước phù hợp với môi trường vườn.

Actuators:

Module 4 relay đóng ngắt các tải công suất lớn gồm đèn, quạt và bơm nước, cách ly hoàn toàn mạch điều khiển với mạch tải để bảo vệ vi điều khiển.

IC shift register 74HC595 được STM32 điều khiển qua GPIO chỉ cần 3 dây tín hiệu để điều khiển 8 ngõ ra song song, tiết kiệm chân GPIO so với nối relay trực tiếp.

Servo SG90 nhận tín hiệu PWM từ STM32 để xoay ESP32-CAM theo chiều ngang và chiều dọc, cho phép người dùng quan sát toàn bộ không gian vườn từ GUI.

Nguồn cấp:

Adapter 12V 5A làm nguồn chính cho toàn hệ thống.

Buck converter XL4016E hạ áp từ 12V xuống 5V để cấp cho các vi điều khiển, module relay và servo.

Nguồn 12V trực tiếp cấp cho các tải công suất lớn như đèn, quạt và bơm.

PHẦN III: THIẾT KẾ VÀ THI CÔNG

CHƯƠNG 1: PHÂN TÍCH YÊU CẦU

1.1. Giới thiệu

Hệ thống vườn thông minh cần đáp ứng các yêu cầu sau: thu thập dữ liệu từ cảm biến (nhiệt độ, độ ẩm, ánh sáng, mực nước) và điều khiển tải (đèn, quạt, bơm) qua STM32 và ESP32, phát triển Server trên Raspberry Pi 5 làm trung tâm xử lý và lưu trữ, cung cấp WebSocket cho IoT và GUI, thiết kế giao diện trực quan cho phép giám sát, xem camera, điều khiển thủ công/tự động và cấu hình ngưỡng, hỗ trợ kết nối từ xa qua Tailscale VPN, đảm bảo phản hồi nhanh, ổn định và có cơ chế phục hồi khi mất kết nối.

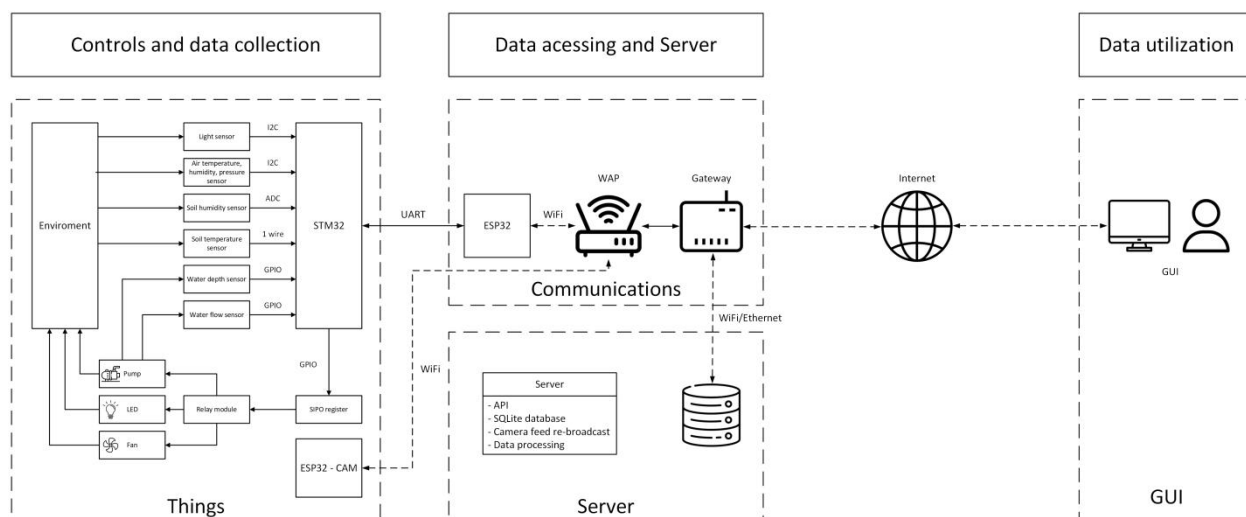
1.2. Mô tả hệ thống

Hệ thống gồm 3 nút chính: nút MCU, nút Server và nút GUI.

Nút MCU: Gồm STM32F103C8T6 và ESP32. STM32 thu thập dữ liệu cảm biến và điều khiển tải qua relay. ESP32 giao tiếp với STM32 qua UART, kết nối WiFi và trao đổi dữ liệu với Server qua WebSocket.

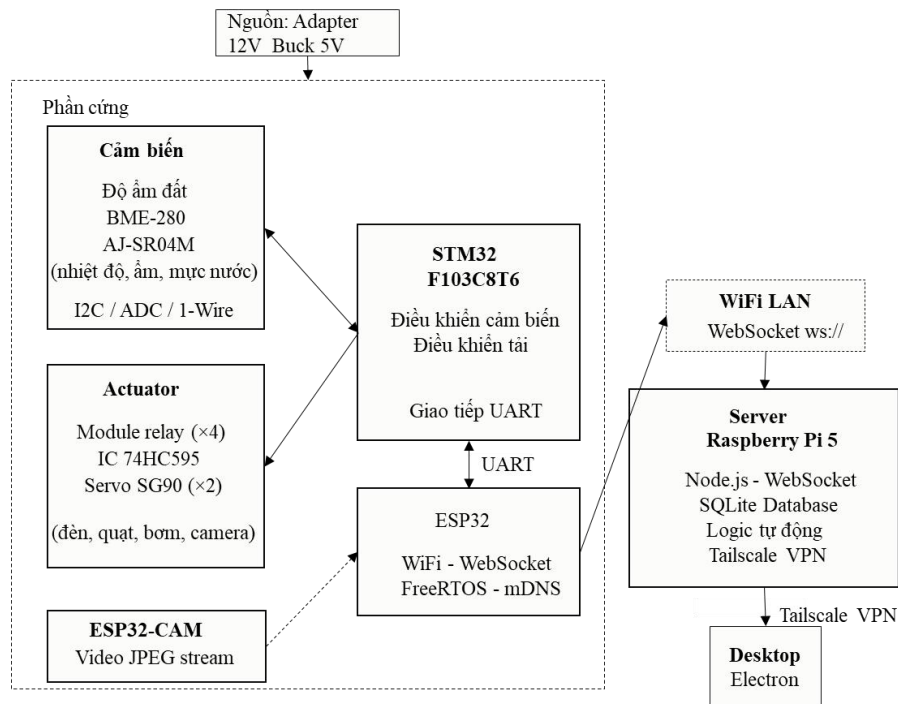
Nút Server: Chạy trên Raspberry Pi 5 sử dụng Node.js, quản lý WebSocket, cơ sở dữ liệu SQLite và logic điều khiển tự động. Server là bộ nhớ chung của hệ thống, xử lý luồng hình ảnh từ ESP32-CAM và chuyển tiếp đến GUI.

Nút GUI: Giao diện người dùng hiển thị dữ liệu cảm biến, video camera, trạng thái thiết bị theo thời gian thực. Cho phép gửi lệnh điều khiển và thay đổi cấu hình qua Tailscale VPN.



Hình 3.1: Sơ đồ hoạt động của hệ thống

1.3. Nguyên tắc làm việc



Hình 3.2: Sơ đồ khối hệ thống

Hệ thống hoạt động theo hai chế độ: tự động dựa trên ngưỡng cảm biến cấu hình sẵn, hoặc thủ công từ GUI. Toàn bộ dữ liệu và trạng thái thiết bị được lưu trên Database của Server và đồng bộ đến GUI qua WebSocket.

STM32 thu thập dữ liệu cảm biến và trả về cho ESP32 qua UART khi được yêu cầu. ESP32 định kỳ lấy dữ liệu từ STM32 và đẩy lên Server, đồng thời nhận lệnh từ Server, chuyển tiếp đến STM32 thực thi và xác nhận kết quả.

Lệnh điều khiển được xử lý qua hàng đợi trong Database - Server ghi lệnh vào bảng commands, ESP32 nhận và thực thi, sau đó xác nhận để Server xóa lệnh. Cơ chế này đảm bảo không mất lệnh kể cả khi ESP32 tạm thời mất kết nối.

1.4. Lựa chọn giải pháp

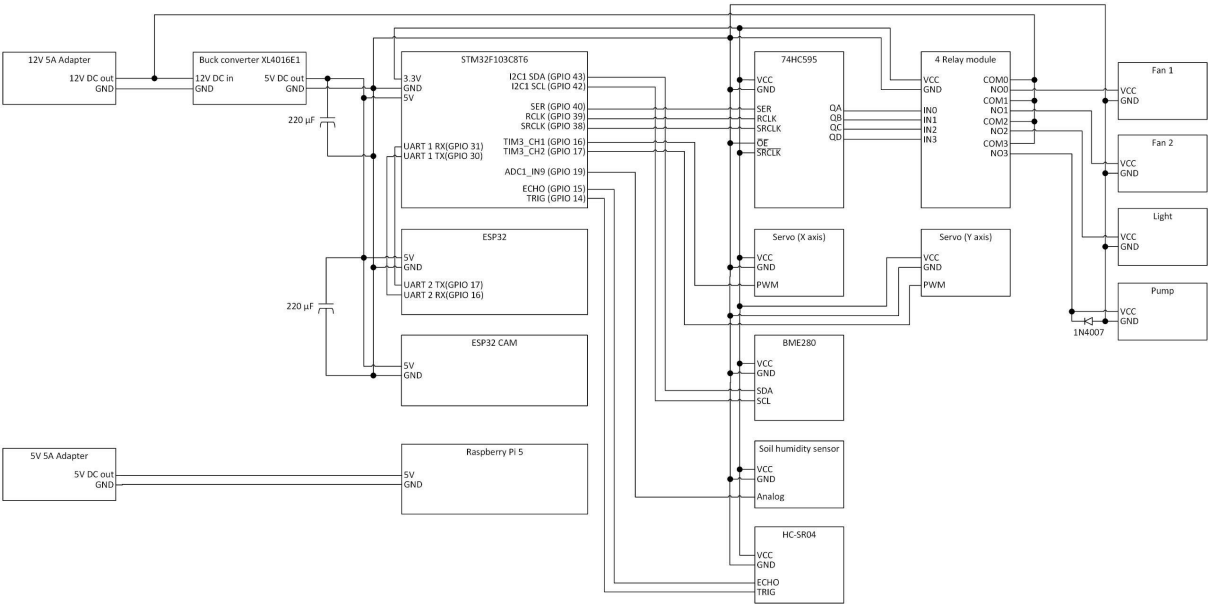
Nốt MCU: Sử dụng STM32F103C8T6 làm vi điều khiển chính thu thập cảm biến và điều khiển tải qua relay và IC 74HC595. ESP32 đóng vai trò module mạng giao tiếp với Server qua WebSocket và kết nối với STM32 qua UART.

Nốt Server: Raspberry Pi 5 chạy Node.js làm trung tâm xử lý, SQLite lưu trữ dữ liệu, Tailscale VPN cho phép truy cập từ xa an toàn mà không cần port forwarding.

Nốt GUI: Dashboard cho phép giám sát dữ liệu thời gian thực, xem camera, điều khiển thủ công và cấu hình tự động với ngưỡng linh hoạt.

CHƯƠNG 2: ỨNG DỤNG

2.1. Phần cứng



Hình 3.3: Sơ đồ kết nối tổng thể phần cứng

Các thiết bị trong hệ thống được chia thành bốn khối:

Khối cảm biến: BME-280 (I2C), cảm biến độ ẩm đất (ADC) và AJ-SR04M đo mực nước tất cả kết nối với STM32.

Khối actuator: Module 4 relay điều khiển đèn, quạt và bơm qua IC 74HC595 (SPI), và 2 servo SG90 xoay ESP32-CAM - STM32 điều khiển toàn bộ.

Khối truyền thông: ESP32 giao tiếp với STM32 qua UART và với Server qua WebSocket. ESP32-CAM truyền video JPEG đến Server qua WebSocket.

Khối nguồn: Adapter 12V 5A cấp cho các tải lớn, buck converter XL4016E hạ xuống 5V cho vi điều khiển và relay, LDO tích hợp trên vi điều khiển tạo 3,3V cho cảm biến và IC logic.

Bảng 3.1: Danh sách linh kiện của hệ thống vườn thông minh

Nhóm	Tên	Số lượng	Chức năng
Nguồn	Adapter 12V 5A	1	Cấp nguồn cho hệ thống
	Buck converter	1	Chuyển 12V thành 3.3V để vừa cấp nguồn cho đèn, quạt, bơm vừa cấp nguồn cho các vi điều khiển và cảm biến
Sensor	Đầu dò độ ẩm đất	1	Cảm biến độ ẩm đất

Nhóm	Tên	Số lượng	Chức năng
	BME-280	1	Cảm biến nhiệt độ, độ ẩm, áp suất không khí
	AJ-SR04M	1	Cảm biến siêu âm đo độ sâu nước
	ESP32 - CAM	1	Cung cấp video feed
Actuator	Module 4 relay	1	Điều khiển thiết bị
	74HC595N	1	Điều khiển module relay
	Servo SG90	2	Điều chỉnh góc quay của ESP32 CAM
	Đèn LED	1	Chiếu sáng cho cây
	Quạt	2	Giúp làm mát hoặc giảm độ ẩm không khí
	Bơm nước	1	Bơm nước tưới cây từ bể chứa nước
Server	Raspberry Pi 5	1	Database, xử lý dữ liệu

2.2. Triển khai bằng phần mềm và các công cụ

2.2.1. STM32CubeIDE

STM32CubeIDE là IDE chính thức của STMicroelectronics, tích hợp sẵn STM32CubeMX cho phép cấu hình đồ họa các ngoại vi và tự động sinh mã khởi tạo HAL. Trong dự án, STM32CubeIDE được dùng để lập trình toàn bộ firmware cho STM32F103C8T6, bao gồm cấu hình UART giao tiếp với ESP32, I2C đọc BME-280, SPI điều khiển IC 74HC595 và GPIO điều khiển relay, servo SG90.

2.2.2. PlatformIO

PlatformIO là extension phát triển nhúng trên Visual Studio Code, hỗ trợ quản lý thư viện và board theo từng project qua file platformio.ini. Trong dự án, PlatformIO được dùng để lập trình firmware cho ESP32 và ESP32-CAM, quản lý các thư viện WiFi, WebSocket, FreeRTOS và mDNS mà không cần cài đặt thủ công.

2.2.3. Node.js

Node.js được dùng để xây dựng Server chạy trên Raspberry Pi 5, triển khai dưới dạng systemd service để tự khởi động cùng hệ thống. Các thư viện chính gồm ws quản lý ba WebSocket endpoint (/iot_nodes, /ui_clients, /cam), sqlite3 truy vấn cơ sở dữ liệu và multicast-dns phát quảng bá địa chỉ IP trong mạng LAN.

2.2.4. SQLite

SQLite được dùng làm cơ sở dữ liệu trung tâm của Server với ba bảng: sensors lưu lịch sử cảm biến theo cơ chế FIFO tối đa 129.600 bản ghi, devices lưu trạng thái hiện tại các tải, và commands đóng vai trò hàng đợi lệnh giữa GUI và ESP32.

2.2.5. Tailscale

Tailscale được cài trên Raspberry Pi 5 và các thiết bị chạy GUI, cho phép kết nối WebSocket đến Server từ bất kỳ mạng nào mà không cần cấu hình port forwarding.

2.2.6. Electron

Electron được dùng để xây dựng ứng dụng desktop GUI trên Windows, đóng gói thành file .exe qua electron-builder. Giao diện gồm bốn tab: Dashboard hiển thị dữ liệu cảm biến, Controls điều khiển thiết bị, Camera xem video và Settings cấu hình ngưỡng tự động.

2.3. Cơ chế giao tiếp

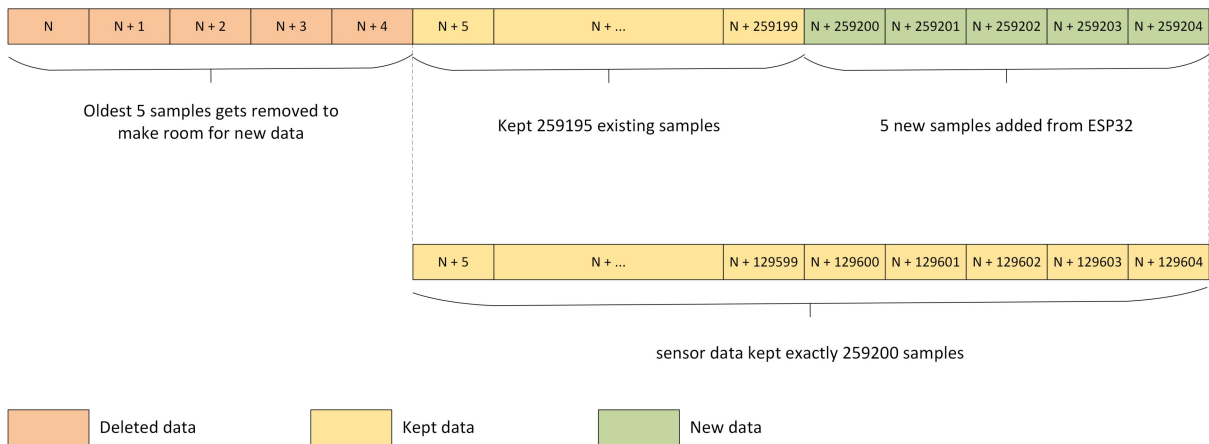
2.3.1. Giao thức giao tiếp giữa các thành phần

Bảng 3.9: Các kiểu gói tin giao tiếp Server - Client của hệ thống

Endpoint	Loại message	Hướng gửi => nhận	Ý nghĩa
/iot_nodes	sensors.insert	ESP32 => Server	Gửi 1 dữ liệu cảm biến đến Server
/iot_nodes	devices.update	ESP32 => Server	Cập nhật trạng thái của 1 tải
/iot_nodes	commands.new	Server => ESP32	Gửi lệnh cần thực thi
/iot_nodes	commands.complete	ESP32 => Server	Xác nhận thực thi lệnh thành công
/ui_clients	sensors.get	GUI => Server	Yêu cầu dữ liệu cảm biến từ Server
/ui_clients	sensors.data	Server => GUI	Gửi dữ liệu cảm biến đến GUI
/ui_clients	devices.get	GUI => Server	Yêu cầu trạng thái hiện tại của các tải từ Server
/ui_clients	devices.data	Server => GUI	Gửi trạng thái hiện tại của các tải đến GUI
/ui_clients	commands.insert	GUI => Server	Gửi một lệnh mới đến table “commands”

/ui_clients	camera.settings	GUI => Server	Gửi cấu hình camera từ GUI đến Server
/cam	camera.settings	Server => ESP32 CAM	Gửi cấu hình camera từ Server đến ESP32 CAM
/cam	binary frame	ESP32 CAM => Server	Gửi frame binary JPEG đến Server
/cam	binary frame	Server => GUI	Gửi frame binary JPEG đến tất cả các client GUI

2.3.2. Quản lý dung lượng Database bằng cơ chế FIFO

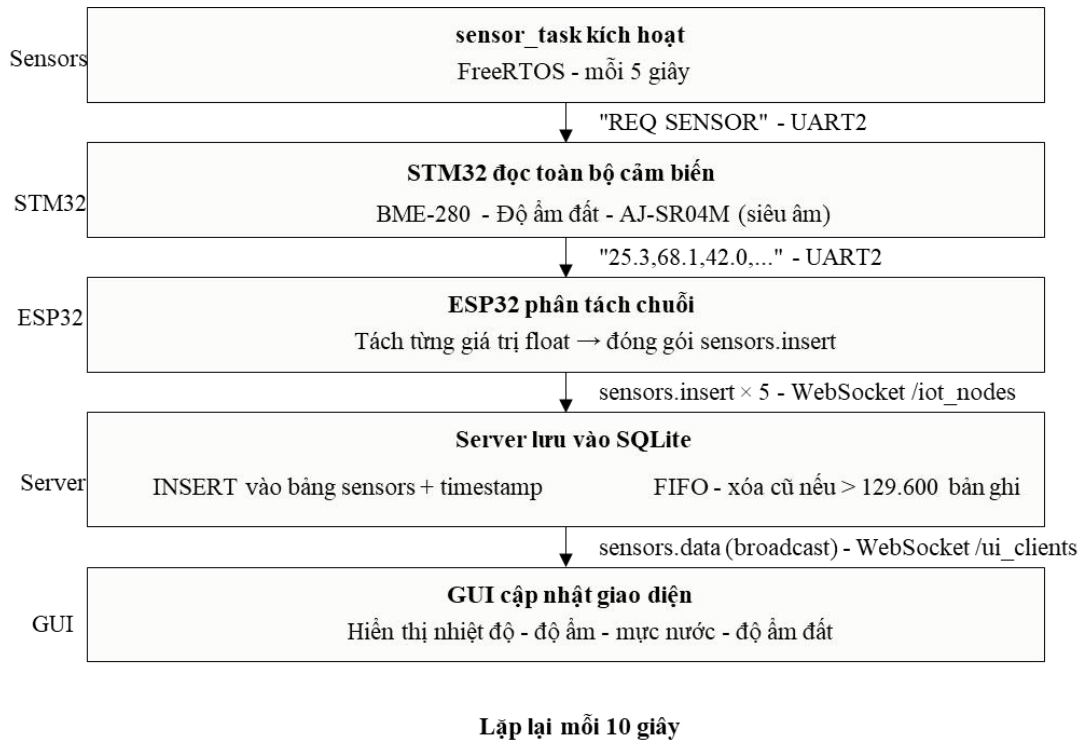


Hình 3.4: Sơ đồ hoạt động quản lý dữ liệu cảm biến trên Database

ESP32 gửi 5 dữ liệu cảm biến mỗi 5 giây, tạo 17280 mẫu/ngày cho mỗi cảm biến. Nếu không kiểm soát, bảng sensors sẽ tăng vô hạn và đầy bộ nhớ. Giới hạn tối đa 259200 mẫu (tương đương 5 cảm biến trong 3 ngày). Sau mỗi lần chèn, Server kiểm tra số lượng, nếu vượt ngưỡng, tự động xóa bản ghi cũ nhất theo cơ chế FIFO. Nhờ vậy, database luôn lưu 3 ngày dữ liệu gần nhất để hiển thị biểu đồ lịch sử mà không cần bảo trì thủ công.

2.4. Các luồng hoạt động

2.4.1. Thu thập và hiển thị dữ liệu cảm biến

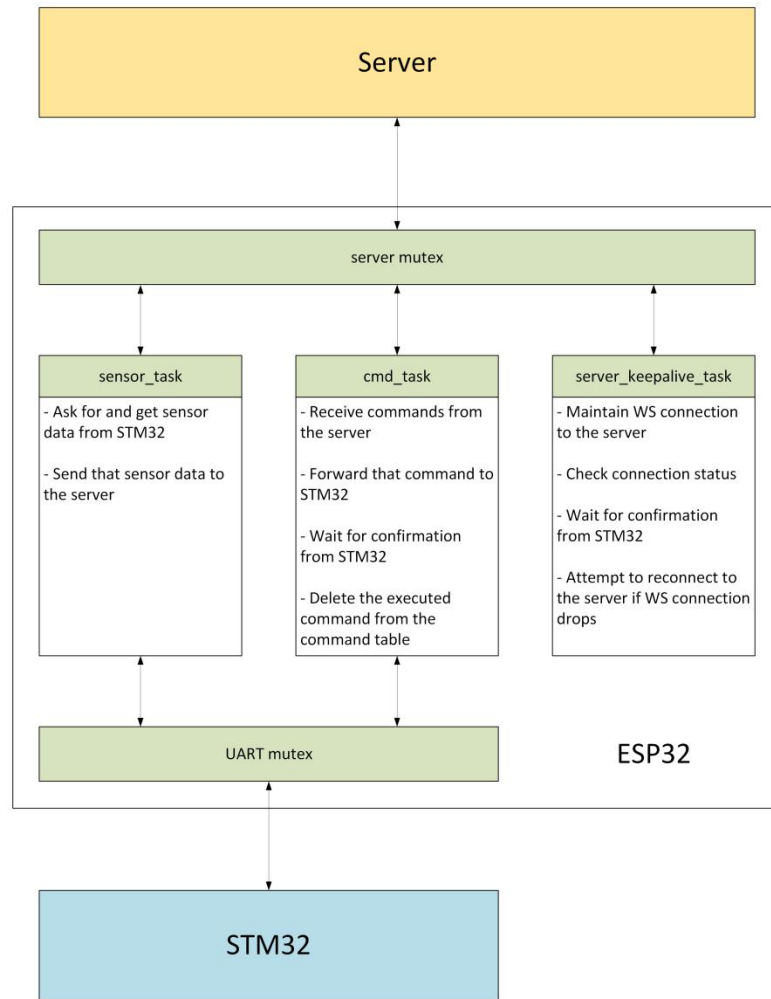


Hình 3.5: Sơ đồ hoạt động thu thập dữ liệu cảm biến

ESP32 gửi lệnh REQ SENSOR đến STM32 qua UART mỗi 10 giây thông qua sensor_task của FreeRTOS. STM32 sau khi nhận lệnh sẽ đọc giá trị từ tất cả cảm biến và trả về cho ESP32 một chuỗi dữ liệu có định dạng cố định gồm các giá trị float ngăn cách bởi dấu phẩy. ESP32 phân tách chuỗi và gửi từng giá trị lên Server qua gói tin sensors.insert.

Server sau khi nhận từng gói sensors.insert sẽ chèn giá trị vào bảng sensors kèm dấu thời gian, đồng thời kiểm tra và xóa các bản ghi cũ nhất nếu tổng số vượt quá 129.600 theo cơ chế FIFO. Các giá trị mới nhất được Server tự động đẩy đến tất cả GUI client đang kết nối qua gói sensors.data, đảm bảo giao diện người dùng luôn phản ánh trạng thái môi trường thực tế.

2.4.2. Gửi và thực thi lệnh điều khiển



Hình 3.6: Sơ đồ các task FreeRTOS và cơ chế chia sẻ tài nguyên

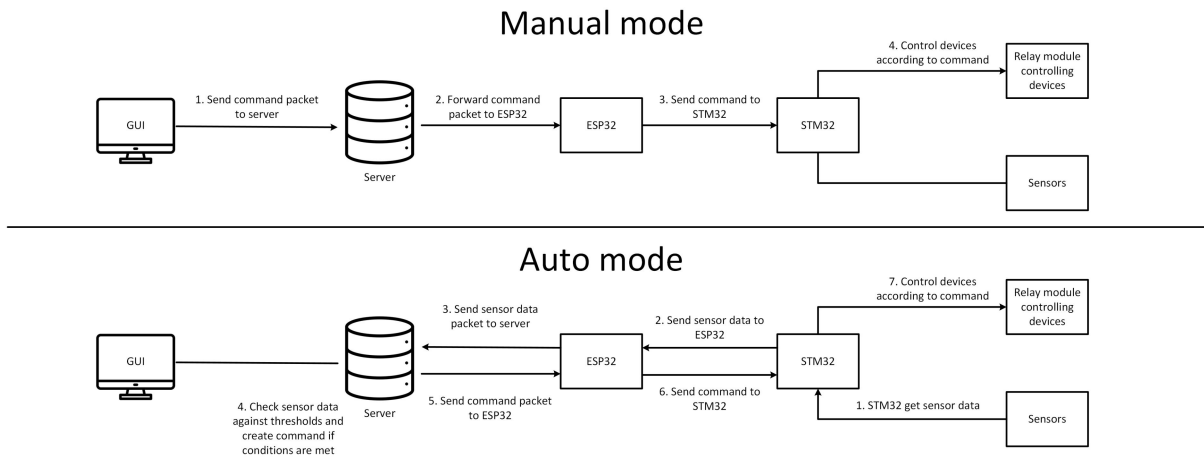
ESP32 sử dụng FreeRTOS chia chương trình thành 3 task độc lập:

- `sensor_task`: Định kỳ gửi lệnh `REQ SENSOR` đến STM32 qua UART mỗi 10 giây, nhận dữ liệu và đẩy lên Server qua gói `sensors.insert`.
- `cmd_task`: Lắng nghe lệnh `commands.new` từ Server, chuyển tiếp đến STM32 định dạng "`<cmd> <value>\r\n`", chờ phản hồi "`EXE OK`" và gửi `commands.complete` xác nhận hoàn thành. Cơ chế timeout tránh kẹt lệnh.
- `server_keepalive_task`: Kiểm tra kết nối WebSocket, nếu mất thì truy vấn mDNS để lấy IP mới của Server và tự động kết nối lại.

Mutex bảo vệ UART và WebSocket khỏi xung đột, Event Group đồng bộ trạng thái WiFi, ESP32 tự khởi động lại nếu mất kết nối quá số lần cho phép.

Khi GUI gửi lệnh (`commands.insert`), Server ghi vào bảng `commands` và chuyển tiếp `commands.new` đến ESP32. ESP32 chuyển đến STM32, chờ "`EXE OK`", gửi `commands.complete` để Server xóa lệnh và cập nhật trạng thái. Nếu ESP32 ngoại tuyến, lệnh vẫn lưu trong hàng đợi và tự động gửi khi kết nối lại.

2.4.3. Điều khiển tự động và thủ công



Hình 3.7: Sơ đồ hoạt động của chế độ tự động và thủ công

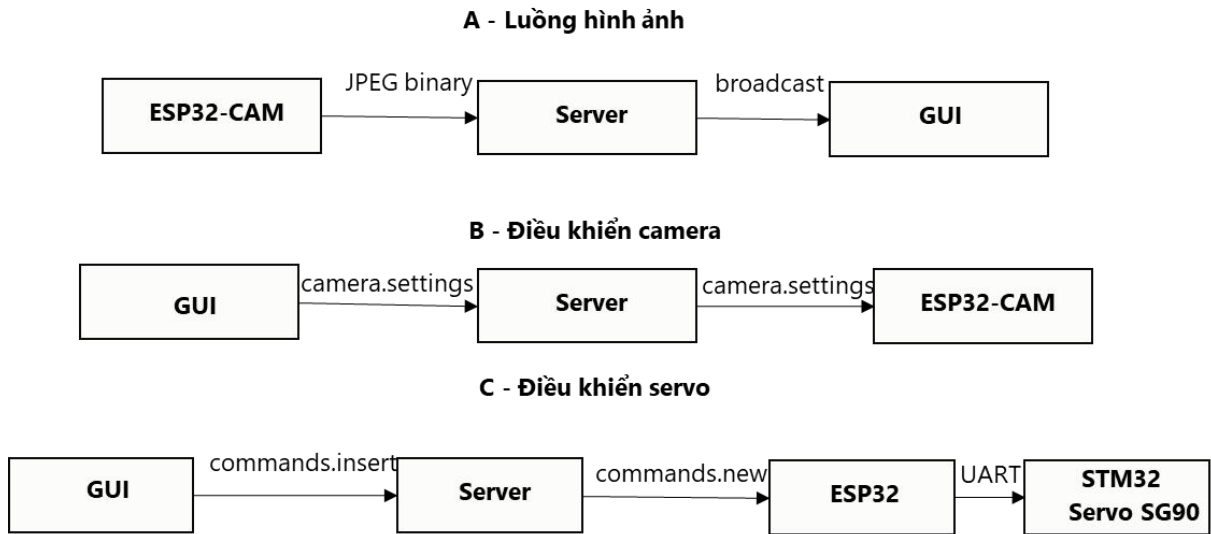
Hệ thống hỗ trợ hai chế độ điều khiển, được chuyển đổi thông qua gói tin config.update từ GUI. Khi GUI gửi yêu cầu chuyển chế độ, Server lưu cấu hình mới vào bộ nhớ và đồng bộ trạng thái đến tất cả GUI client đang kết nối, đồng thời kích hoạt lại hàm đánh giá điều khiển để hệ thống phản ứng ngay.

Ở chế độ tự động, Server liên tục đánh giá dữ liệu cảm biến mới nhất theo các ngưỡng đã cấu hình:

- Quạt 1: Bật khi nhiệt độ vượt ngưỡng nhiệt độ tối đa.
- Quạt 2: Bật khi độ ẩm không khí vượt ngưỡng độ ẩm tối đa.
- Đèn LED: Bật/tắt theo khung thời gian đã cấu hình sẵn.
- Bơm nước: Bật khi độ ẩm đất dưới ngưỡng tối thiểu và mực nước trong bồn chứa đủ, tự động tắt khi một trong hai điều kiện không còn thỏa mãn.

Ở chế độ thủ công, logic tự động bị vô hiệu hoá và người dùng điều khiển trực tiếp từng thiết bị qua tab Controls của GUI. Các lệnh được gửi theo cùng cơ chế hàng đợi lệnh.

2.3.4. Giám sát hình ảnh và điều khiển camera

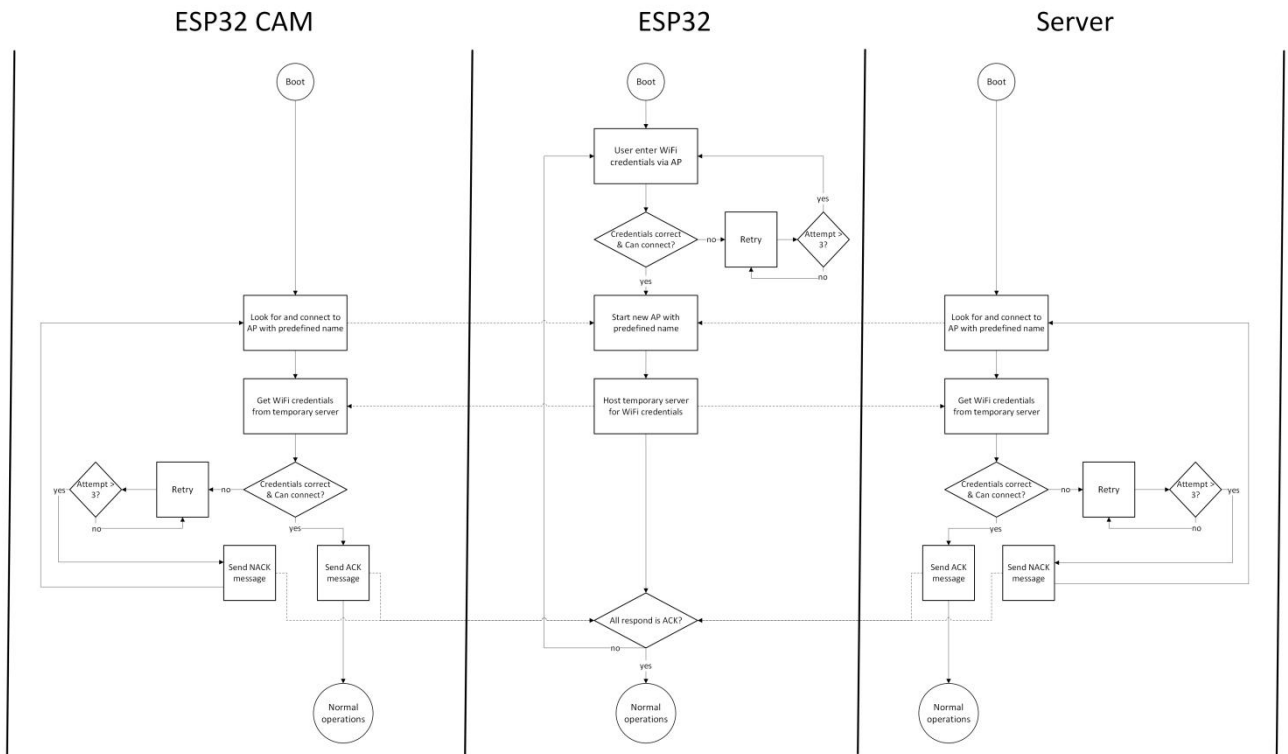


Hình 3.8: Các luồng làm việc với camera

ESP32-CAM liên tục chụp frame JPEG và gửi đến Server dưới dạng binary qua endpoint WebSocket /cam ở tốc độ tối đa 15 FPS. Server lưu frame mới nhất vào bộ nhớ tạm và chuyển tiếp đến tất cả GUI client đang kết nối. Khi một GUI client mới kết nối, Server lập tức gửi frame gần nhất để người dùng không phải chờ frame tiếp theo.

Người dùng có thể điều chỉnh các thông số camera từ tab Camera của GUI bằng cách gửi gói camera.settings đến Server. Server kiểm tra hợp lệ các thông số (framesize, quality, vflip, hmirror, brightness, contrast, saturation) rồi chuyển tiếp đến ESP32-CAM. Ngoài ra, 2 servo SG90 điều khiển góc quay của camera có thể được điều khiển từ GUI thông qua lệnh SRX và SRY, cho phép người dùng quan sát toàn bộ không gian vườn từ xa.

2.3.5. Cấu hình mạng WiFi cho hệ thống

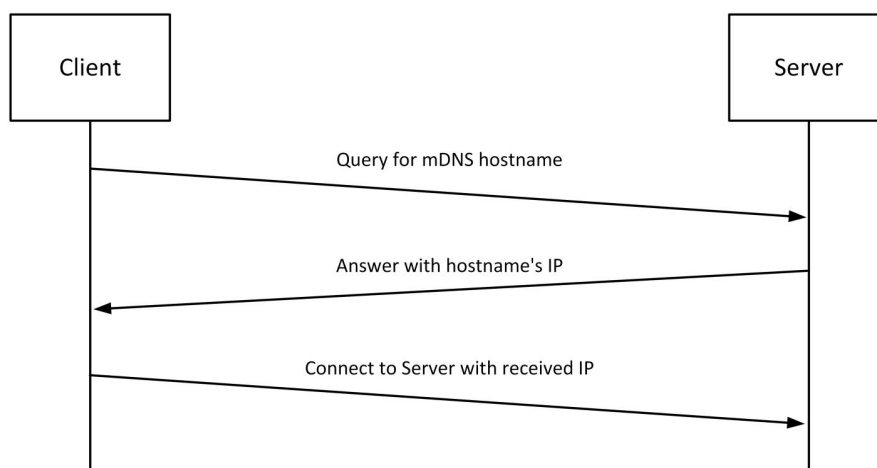


Hình 3.9: Lưu đồ quy trình cấu hình WiFi

Để tránh phải code cứng thông tin mạng WiFi vào firmware, hệ thống có tính năng cho phép người dùng cấu hình mạng WiFi tại chỗ thông qua trình duyệt web mà không cần kết nối cáp hay cài phần mềm bổ sung.

Sau khi khởi động, ESP32 tạo một Access Point (AP1) và host trang HTML để người dùng kết nối và nhập thông tin mạng WiFi đích. Sau khi người dùng xác nhận, ESP32 lưu thông tin WiFi, tắt AP1 và kiểm tra khả năng kết nối với mạng đích. Nếu thành công, ESP32 tạo AP2 để Raspberry Pi (Server) và ESP32-CAM kết nối vào và lấy thông tin WiFi qua endpoint /creds. Sau khi tất cả thiết bị xác nhận kết nối thành công bằng tín hiệu ACK, ESP32 tắt AP2 và toàn bộ hệ thống bắt đầu hoạt động trên cùng mạng WiFi.

2.3.6. Phân giải địa chỉ Server bằng mDNS



Hình 3.10: Cách Client lấy IP của Server qua mDNS

Trong mạng DHCP, IP Server có thể thay đổi, gây mất kết nối client. Thay vì dùng IP tĩnh dễ xung đột, nhóm chọn mDNS là Server đăng ký hostname cố định, client truy vấn multicast để lấy IP hiện tại và tự động kết nối lại khi IP thay đổi. mDNS được triển khai trên Raspberry Pi qua package multicast-dns trong Node.js, và trên ESP32 qua server_keepalive_task khi phát hiện mất kết nối WebSocket.

2.5. Tổng hợp phương án thực hiện đề tài

Từ các phương án và thiết kế đã trình bày, hệ thống vườn thông minh được xây dựng nhằm giúp người dùng giám sát môi trường và điều khiển thiết bị trong vườn một cách tiện lợi và linh hoạt từ xa. Nguyên lý hoạt động của hệ thống được tổng hợp như sau:

- Database SQLite đóng vai trò là bộ nhớ chung và buffer giữa các thành phần.
- FreeRTOS trên ESP32 phân chia các chức năng thành ba task độc lập đảm bảo hệ thống phản hồi nhanh và ổn định.
- Hàng đợi lệnh trong bảng commands đảm bảo không có lệnh nào bị mất.
- Cơ chế FIFO trên bảng sensors giữ database ở kích thước ổn định, lưu trữ đủ 3 ngày dữ liệu lịch sử cho người dùng theo dõi xu hướng môi trường.
- Tailscale VPN cho phép GUI kết nối với Server từ bất kỳ mạng nào.
- Tính năng cấu hình WiFi tại chỗ giúp hệ thống linh hoạt trong việc thay đổi mạng, hữu ích khi cần di chuyển nhiều địa điểm khác nhau.
- ESP32-CAM cung cấp khả năng giám sát trực quan bổ sung, cho phép người dùng quan sát tình trạng thực tế của vườn từ xa.

Tóm lại, mô hình vườn thông minh kết hợp hiệu quả giữa phần cứng nhúng (STM32, ESP32), hạ tầng Server nhúng (Raspberry Pi 5, Node.js, SQLite) và giao diện người dùng (Electron) để tạo ra một hệ thống IoT hoàn chỉnh, có khả năng giám sát và điều khiển từ xa với độ tin cậy cao.

PHẦN IV: KẾT QUẢ

1. Đánh giá độ chính xác cảm biến

Phần đánh giá độ chính xác được thực hiện nhằm xác định mức độ đáp ứng của các cảm biến trong mô hình so với giá trị tham chiếu thực tế, từ đó xem xét độ tin cậy của hệ thống trong quá trình vận hành. Việc đánh giá tập trung vào ba cảm biến chính gồm cảm biến độ ẩm đất, cảm biến nhiệt độ và độ ẩm không khí và cảm biến khoảng cách.

1.1. Đánh giá độ chính xác của cảm biến độ ẩm đất

Giá trị đo được sẽ được đối chiếu với độ ẩm tính theo công thức tham chiếu dựa trên khối lượng mẫu đất ở hai trạng thái khô và ướt. Cụ thể, độ ẩm phần trăm được xác định theo công thức:

$$moisture = \frac{m_{wet} - m_{dry}}{m_{dry}} \times 100$$

Trong đó:

m_{wet} : Khối lượng mẫu đất ở trạng thái ướt

m_{dry} : Khối lượng mẫu đất ở trạng thái khô

Sử dụng hộp chứa đất khô cân được 0.52kg và từ từ đổ thêm nước đến khi hộp chứa đất cân được 0.71 và 0.85, ta có được các mức độ ẩm lý thuyết là 0%, 36.54% và 63.46%.

Kết quả:

Bảng 4.1: Kết quả đo độ ẩm đất so với thực tế

Giá trị thực tế (%)	Lần đo	Giá trị đo được (%)	Sai số tuyệt đối (%)
0%	1	0.96	0.96
	2	0.93	0.93
	3	0.87	0.87
	4	0.79	0.79
	5	0.87	0.87
	Trung bình	0.88	0.88
36.54%	1	36.02	0.52
	2	35.53	1.01
	3	35.66	0.88
	4	35.62	0.92

	5	35.51	1.03
	Trung bình	35.67	0.87
63.46%	1	62.77	0.69
	2	62.70	0.76
	3	62.77	0.69
	4	62.04	1.42
	5	62.11	1.35
	Trung bình	62.48	0.98

Đánh giá:

Sai số tuyệt đối trung bình ở cả ba mức tham chiếu đều ở mức thấp (0.88%, 0.87% và 0.98%), cho thấy cảm biến đạt độ chính xác tương đối tốt so với phương pháp tính độ ẩm theo khối lượng mẫu. Sai số không tăng đáng kể khi độ ẩm thực tế tăng, chứng tỏ cảm biến hoạt động ổn định trên toàn dải đo được khảo sát.

Tuy nhiên, cảm biến độ ẩm đất cần một khoảng thời gian trước khi đạt được giá trị đo chính xác. Như ở lần đo giá trị thực tế 63.46%, lần đầu lấy mẫu cảm biến đo được đến 77.84% nhưng giảm dần đến khoảng thực tế sau khoảng 5 – 10 phút. Ngoài ra, giá trị đọc được bởi cảm biến cũng chịu ảnh hưởng lớn từ vị trí đặt cảm biến vì vị trí được tưới nước không đồng đều dẫn đến giá trị độ ẩm tăng hoặc giảm tùy vào vị trí được đặt.

1.2. Đánh giá độ chính xác của cảm biến nhiệt độ, độ ẩm

Đối với cảm biến nhiệt độ và độ ẩm không khí, dữ liệu thu được sẽ được so sánh với số liệu từ các trang web dự báo thời tiết hoặc nguồn dữ liệu khí tượng trực tuyến tại cùng thời điểm đo.

Tuy nhiên, phương pháp này có một số hạn chế nhất định, do dữ liệu trên website thường được tổng hợp từ khu vực rộng hơn và có thể không phản ánh chính xác điều kiện vi khí hậu tại vị trí đặt cảm biến. Vì vậy, kết quả so sánh chỉ mang tính tương đối và chủ yếu dùng để tham khảo mức độ gần đúng của cảm biến.

Điều kiện đo trong nhà, thời gian 12h – 17h ngày 29/06/2026 tại Bà Rịa – Vũng Tàu. Nguồn dữ liệu thực tế: <https://thoitiet.vn/ba-ria-vung-tau/vung-tau/theo-gio>

Kết quả:

Bảng 4.2: Kết quả đo nhiệt độ so với thực tế

Giá trị thực tế (°C)	Lần đo	Giá trị đo được (°C)	Sai số tuyệt đối (°C)

32.4	1	33.07	0.67
	2	32.99	0.59
	3	33.05	0.65
	4	33.03	0.63
	5	33.03	0.63
	Trung bình	33.03	0.63
31.8	1	32.25	0.45
	2	32.23	0.43
	3	32.26	0.46
	4	32.15	0.35
	5	32.19	0.39
	Trung bình	32.22	0.42
30.3	1	30.52	0.22
	2	30.58	0.28
	3	30.62	0.32
	4	30.53	0.23
	5	30.55	0.25
	Trung bình	30.56	0.26

Bảng 4.1: Kết quả đo độ ẩm không khí so với thực tế

Giá trị thực tế (%)	Lần đo	Giá trị đo được (%)	Sai số tuyệt đối (%)
68%	1	69.24	1.24
	2	69.01	1.01
	3	69.55	1.55
	4	69.22	1.22
	5	69.08	1.08
	Trung bình	69.22	1.22
70%	1	68.99	1.01
	2	69.09	0.91

	3	68.55	1.45
	4	69.53	0.47
	5	69.42	0.58
	Trung bình	69.12	0.88
83%	1	81.19	1.81
	2	81.43	1.57
	3	82.46	1.54
	4	80.25	1.75
	5	81.61	1.39
	Trung bình	81.39	1.61

Đánh giá:

Đối với nhiệt độ, sai số tuyệt đối trung bình dao động trong khoảng 0.26°C – 0.63°C , là mức sai số nhỏ và chấp nhận được cho ứng dụng giám sát môi trường. Đối với độ ẩm không khí, sai số trung bình lớn hơn, trong khoảng 0.88% – 1.53% , một phần xuất phát từ hạn chế của phương pháp đối chiếu: dữ liệu tham khảo được lấy từ website dự báo thời tiết theo khu vực, phản ánh điều kiện trung bình ngoài trời tại Vũng Tàu, trong khi phép đo thực tế được thực hiện trong nhà nên điều kiện khác như luồng không khí, nhiệt độ phòng, độ ẩm cục bộ có thể khác biệt với khu vực ngoài trời mà nguồn dữ liệu ghi nhận. Bên cạnh đó, thời điểm cập nhật số liệu trên website cũng không hoàn toàn trùng khớp với thời điểm đo thực tế, góp phần làm sai số dao động khác nhau giữa các lần đo. Vì vậy, kết quả so sánh độ ẩm chỉ mang tính tương đối, phù hợp để đánh giá xu hướng và mức độ gần đúng hơn là một phép kiểm định chính xác tuyệt đối.

1.3. Đánh giá độ chính xác của cảm biến khoảng cách

Đối với cảm biến khoảng cách, kết quả đo sẽ được đối chiếu trực tiếp bằng thước đo để xác định sai số giữa giá trị hiển thị của cảm biến và khoảng cách thực tế. Đây là phương pháp kiểm tra đơn giản và phù hợp để đánh giá độ chính xác trong phạm vi đo của cảm biến.

Kết quả:

Giá trị thực tế (mm)	Lần đo	Giá trị đo được (mm)	Sai số tuyệt đối (mm)
10	1	10.28	0.28
	2	10.11	0.11

	3	10.11	0.11
	4	10.11	0.11
	5	9.59	0.41
	Trung bình	10.04	0.20
20	1	19.87	0.13
	2	20.04	0.04
	3	20.21	0.21
	4	20.04	0.04
	5	20.21	0.21
	Trung bình	20.07	0.13
30	1	29.84	0.16
	2	30.02	0.02
	3	29.77	0.23
	4	29.67	0.33
	5	30.14	0.14
	Trung bình	29.89	0.18

Đánh giá:

Sai số tuyệt đối trung bình ở cả ba mức đo đều rất nhỏ, lần lượt là 0.20mm, 0.13mm và 0.18mm. Khác với một sai số mang tính hệ thống, giá trị đo ở đây dao động ngẫu nhiên quanh giá trị thực tế. Có lần đo cao hơn, có lần đo thấp hơn ngay trong bảng đo trên và cả trong hoạt động thực tế cho thấy không tồn tại độ lệch offset cố định như thường thấy ở các cảm biến chưa hiệu chỉnh. Với độ lệch nhỏ và tính ổn định qua nhiều lần đo, cảm biến khoảng cách trong phạm vi 10 - 30mm cho kết quả đáng tin cậy, đáp ứng tốt yêu cầu sử dụng.

2. Hoạt động của hệ thống

2.1. Quá trình boot và chọn mạng ban đầu của hệ thống



Hình 4.1: Mạng do ESP32 tạo

Sau khi nguồn cho hệ thống, một mạng WiFi sẽ xuất hiện mang tên “For God so loved the world” sẽ xuất hiện. Đây là AP được tạo ra bởi ESP32 để người dùng có thể chọn mạng WiFi và nhập mật khẩu cho mạng đó. Tiến hành kết nối vào mạng rồi sử dụng trình duyệt truy cập vào IP 10.0.0.1 để chọn WiFi cho hệ thống.

⛔ Không có kết nối Internet

🏠 10.0.0.1 + 11 ⋮

WiFi Setup

SSID

Xom Tro (-77 dBm) ▾

Password

Save

Hình 4.2: Nhập mật khẩu mạng

Từ trang HTML này, người dùng có thể tiến hành chọn mạng WiFi và mật khẩu. Sau khi đã chọn xong, nhấn “Save” để lưu lựa chọn này. ESP32 sẽ tự động tắt AP này và mở một AP mới để server và ESP32 CAM có thể truy cập và lấy SSID và mật khẩu vừa nhập.

```
[BOOT] AP conf2: http://10.0.0.1
[CONF2] CAM ACK
[CONF2] SERVER ACK
[SYNC] All devices connected
```

Hình 4.3: ESP32 nhận được hai phản hồi ACK từ CAM và Server

Sau khi server và ESP32 CAM đã lấy và thử mạng WiFi này xong và thành công, chúng sẽ gửi gói tin ACK về cho ESP32. Sau khi đã nhận đủ 2 gói tin này, ESP32 tắt AP vừa tạo rồi kết nối vào WiFi được chọn trước đó.

```
[WIFI] Reconnecting to Xom Tro
.....
[WiFi] IP: 192.168.1.38
-
[WIFI] Connected, IP: 192.168.1.38
```

Hình 4.4: ESP32 tiến hành kết nối với mạng WiFi được chọn

Để kết nối với server, ESP32 và ESP32 CAM phải lấy được địa chỉ IP của server bằng cách sử dụng mDNS và server sẽ gửi địa chỉ IP của mình đến 2 thiết bị này. Sau khi đã giải được mDNS sang địa chỉ IP thực tế, hệ thống có thể bắt đầu hoạt động.

```
[mDNS] failed to resolve gloriainexcelsisdeo
[mDNS] gloriainexcelsisdeo -> 192.168.1.40
```

Hình 4.5: ESP32 phân giải địa chỉ của Server với mDNS

2.2. Giá trị cảm biến được gửi lên server và đến GUI

Sau mỗi 5 giây, ESP32 sẽ yêu cầu các giá trị cảm biến từ STM32, sau đó các giá trị cảm biến này sẽ được ESP32 gửi đến server và được lưu vào table “sensors” của database.

Ở đây, ESP32 yêu cầu dữ liệu cảm biến từ STM32 và STM32 trả về các dữ liệu lần lượt cho các giá trị độ ẩm đất, độ sâu của nước, nhiệt độ, áp suất (không sử dụng trong GUI) và độ ẩm không khí.

```
[SENSOR TASK] Serial2 SENT: REQ SENSOR
[SENSOR TASK] STM RESPOND: 61.25,4.78,33.51,1006.88,61.37
[SENSOR TASK] DATA PUSHED TO SERVER
```

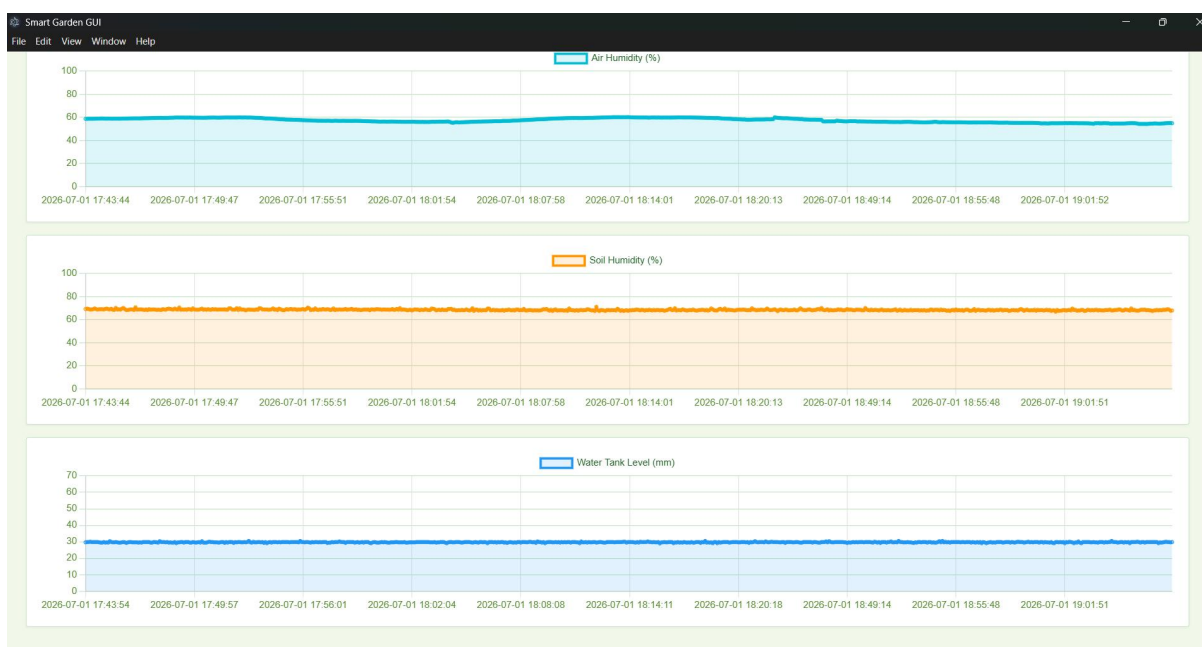
Hình 4.6: ESP32 yêu cầu dữ liệu cảm biến và gửi lên Server

Sau khi các dữ liệu này đã được ESP32 gửi lên server, GUI sẽ yêu cầu các dữ liệu mới này từ server để hiển thị lên tab “Dashboard”.



Hình 4.7: Các giá trị vừa được ESP32 đẩy lên Server hiển thị trên GUI

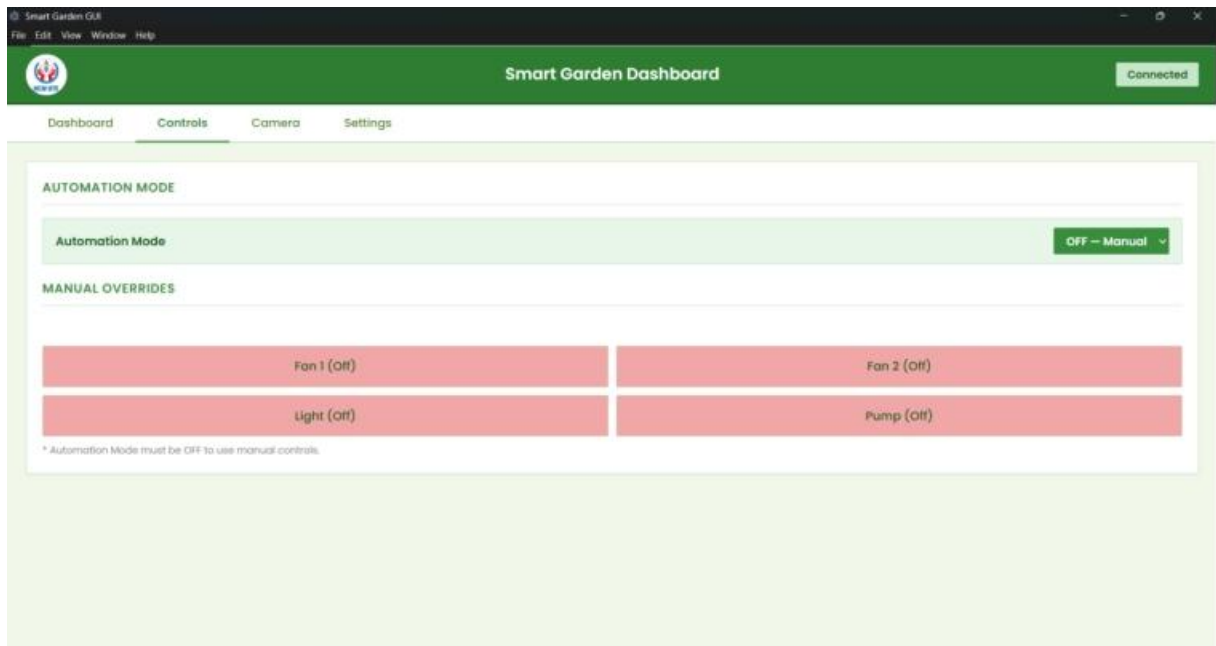
Toàn bộ dữ liệu cảm biến sau khi được lưu vào bảng "sensors" cũng đóng vai trò làm nguồn dữ liệu lịch sử phục vụ cho việc vẽ đồ thị xu hướng theo thời gian trên GUI, cho phép người dùng không chỉ theo dõi giá trị tức thời mà còn quan sát được diễn biến môi trường vườn trong một khoảng thời gian dài, từ đó đưa ra quyết định canh tác phù hợp hơn.



Hình 4.8: Dữ liệu cảm biến trong database được dùng để xây dựng biểu đồ

2.3. Điều khiển các thiết bị đèn, quạt, bơm và các servo từ GUI ở chế độ thủ công

Từ GUI, người dùng có thể điều khiển các thiết bị đèn, quạt và bơm từ tab “Controls”. Ở đây, để điều khiển các thiết bị này một cách thủ công người dùng phải tắt chế độ thủ công ở lựa chọn “Automation mode”. Khi đó các nút điều khiển bên dưới sẽ sáng lên và người dùng có thể nhấn các nút này để đổi trạng thái của thiết bị tương ứng.



Hình 4.9: Giao diện tab điều khiển thiết bị



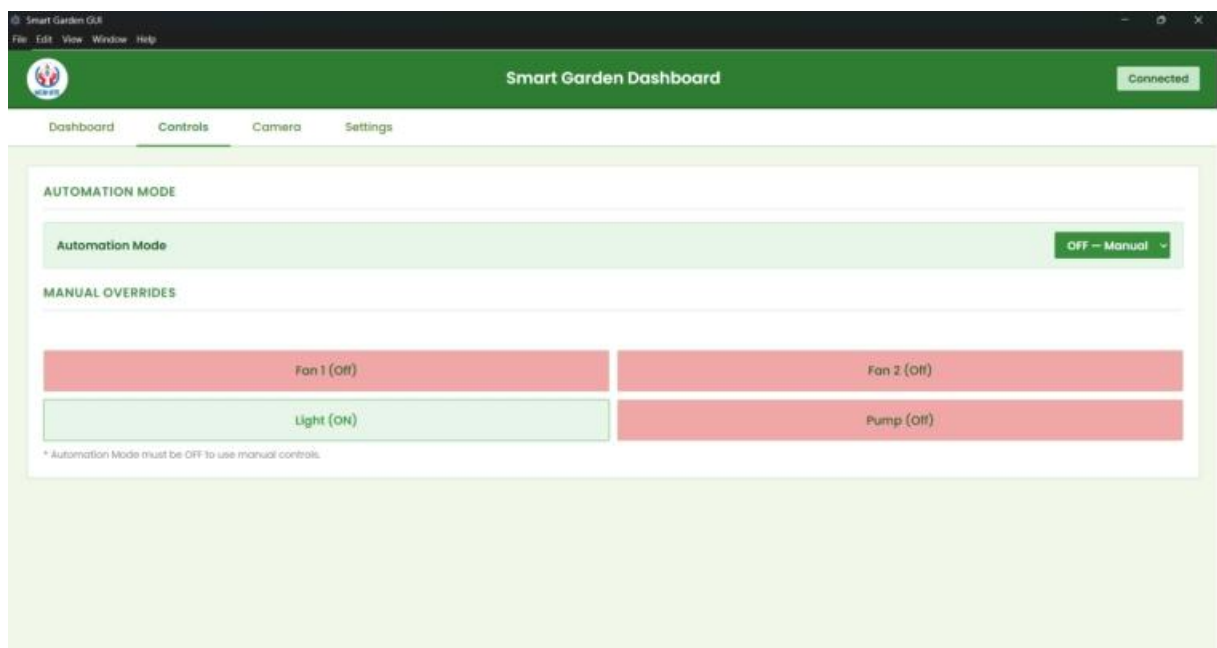
Hình 4.10: Hệ thống khi chưa bật biết bị

Ở đây, tất cả các thiết bị đều tắt. Để bật đèn lên ta nhấn nút “Light” và GUI sẽ gửi gói tin JSON đưa lệnh này vào hàng chờ xử lý lệnh. Server sau đó sẽ gửi lệnh này từ trong hàng chờ xử lý lệnh xuống ESP32 và ESP32 sẽ gửi lệnh này đến STM32 để xử lý. Khi đó relay sẽ đóng lại và đèn sẽ sáng.

```
[CMD TASK] Received command id=1293 cmd=3FFCE778
[CMD TASK] Serial2 SENT: REQ CMD
[CMD TASK] STM RESPOND: ACK
[CMD TASK] Serial2 SENT: RLY 4
[CMD TASK] STM RESPOND: EXE OK
```

Hình 4.11: ESP32 nhận lệnh bật đèn từ Server

Sau đó, trạng thái đèn bật này cũng được cập nhật trên GUI thành ON khi trước đó nó là OFF.

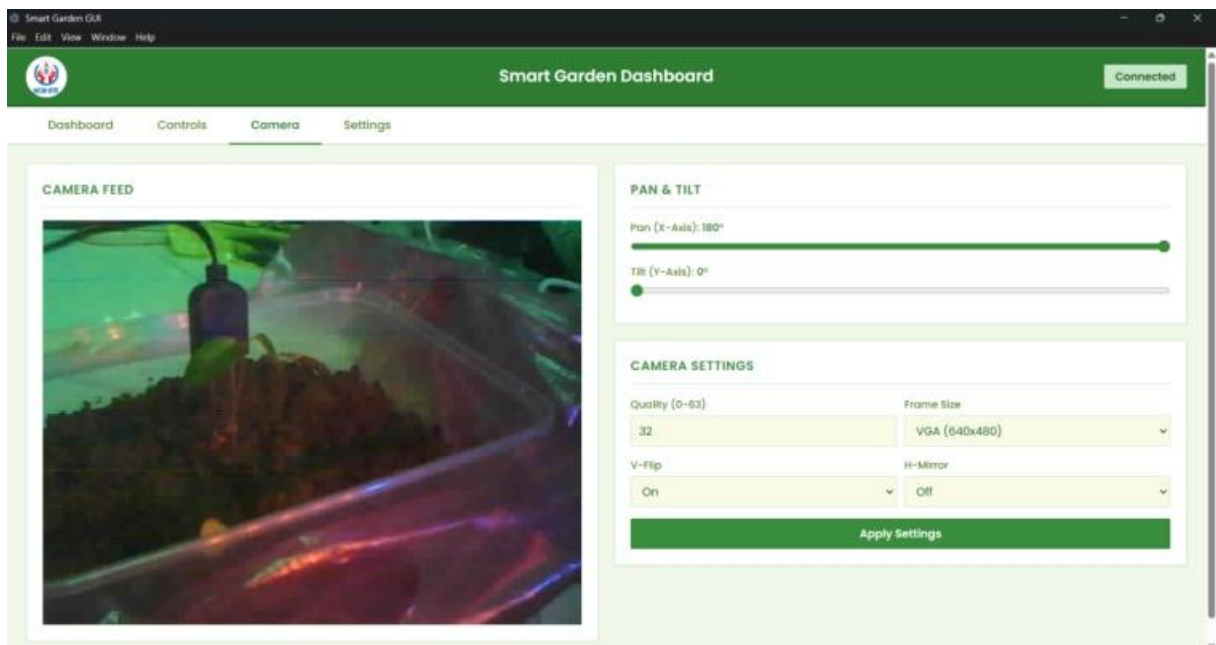


Hình 4.12: Nhấn nút Light(ON) để bật đèn



Hình 4.13: Đèn của hệ thống đã bật

Đối với các servo, người dùng cần phải đi đến tab “Camera” để có thể điều khiển nhằm tạo sự thuận tiện trong quá trình sử dụng giúp người dùng có thể vừa quan sát vị trí của camera vừa di chuyển nó mà không phải chuyển đổi qua lại giữa 2 tab.



Hình 4.14: Giao diện tab điều khiển Camera

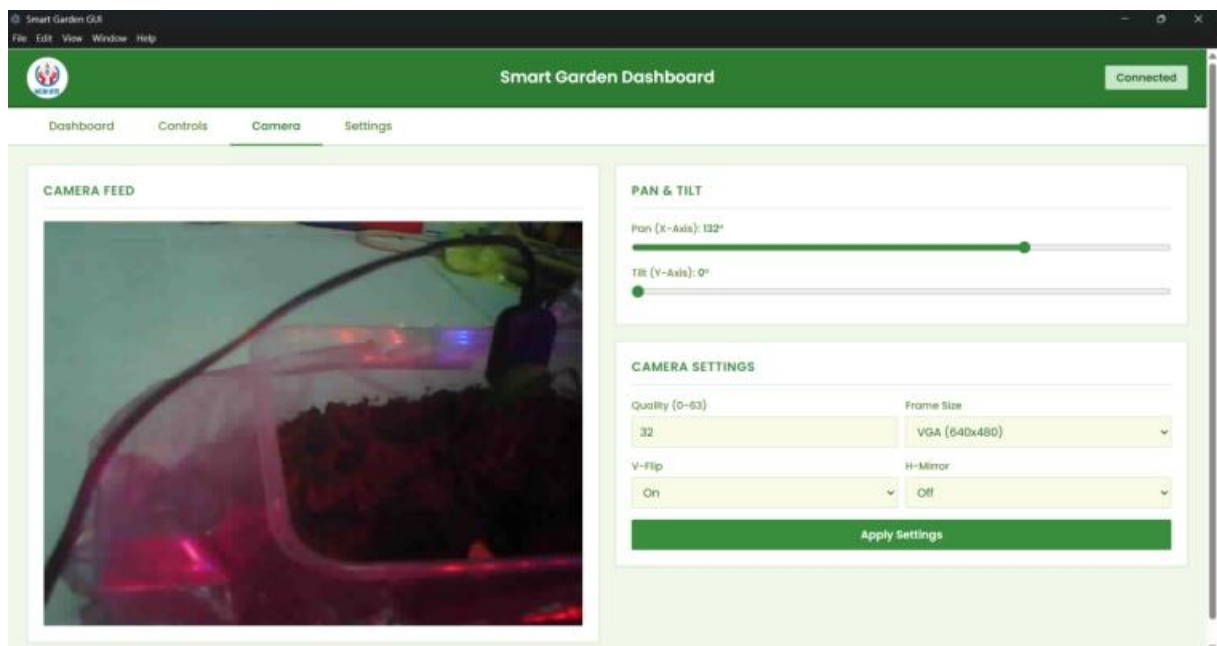
Ở hình, vị trí hiện tại của hai servo x và y lần lượt là 180 độ và 0 độ, camera khi đó đang quay lại vị trí cây trồng. Để thay đổi vị trí của hai servo này, người dùng thực hiện thao tác kéo thả trên 2 thanh trượt tương ứng với servo muốn điều khiển. Tương tự như đối với các thiết bị đèn, quạt, bơm GUI sẽ gửi gói tin JSON đến server vào

hàng chờ lệnh. Các lệnh điều khiển servo này sau đó được gửi xuống ESP32 và ESP32 sẽ tiến hành gửi các lệnh này đến STM32 để thực thi.

Nếu trong trường hợp này người dùng muốn quan sát cây trồng ở một góc độ khác như với servo x ở 132 độ và servo y ở 8 độ, tiến hành kéo thả lần lượt hai thanh trượt điều khiển hai servo này. Các servo sẽ ngay lập tức nhận được lệnh từ GUI khi người dùng thả vị trí mong muốn của servo mà không cần nhấn bất kỳ nút apply nào.

```
[CMD TASK] Received command id=1296 cmd=3FFCE778
[CMD TASK] Serial2 SENT: REQ CMD
[CMD TASK] STM RESPOND: ACK
[CMD TASK] Serial2 SENT: SRX 132
[CMD TASK] STM RESPOND: EXE OK
```

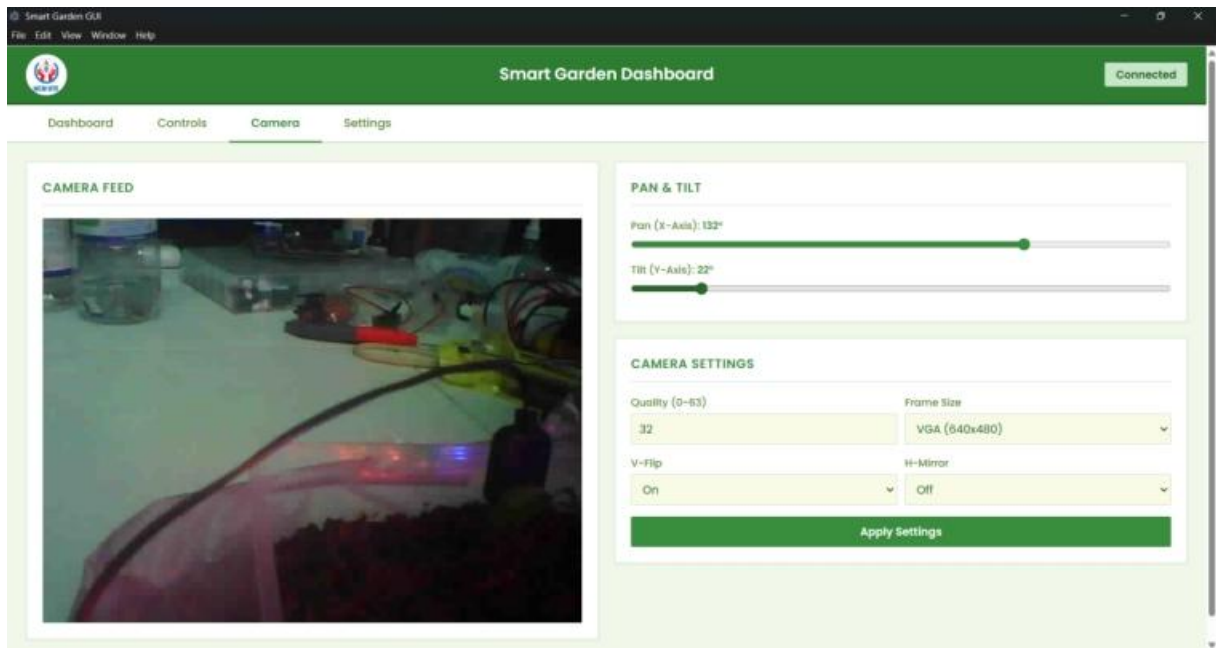
Hình 4.15: ESP32 nhận lệnh điều khiển servo x



Hình 4.16: Đặt góc quay tùy chỉnh

```
[CMD TASK] Received command id=1298 cmd=3FFCE778
[CMD TASK] Serial2 SENT: REQ CMD
[CMD TASK] STM RESPOND: ACK
[CMD TASK] Serial2 SENT: SRY 22
[CMD TASK] STM RESPOND: EXE OK
```

Hình 4.17: ESP32 nhận lệnh điều khiển servo y



Hình 4.18: Thay đổi góc quay - động cơ quay theo

Đối với việc điều khiển servo bằng thao tác kéo thả, hệ thống không gửi lệnh liên tục theo từng pixel di chuyển của thanh trượt mà chỉ gửi lệnh khi người dùng thả chuột tại vị trí mong muốn. Cách tiếp cận này giúp giảm đáng kể số lượng gói tin JSON được gửi đi trong một lần điều chỉnh góc camera, tránh làm nghẽn hàng chờ lệnh cũng như giảm tải xử lý không cần thiết cho ESP32 và STM32, trong khi vẫn đảm bảo trải nghiệm điều khiển camera diễn ra tức thời và trực quan đối với người dùng.

2.4. Điều khiển các thiết bị đèn, quạt, bơm từ GUI ở chế độ tự động

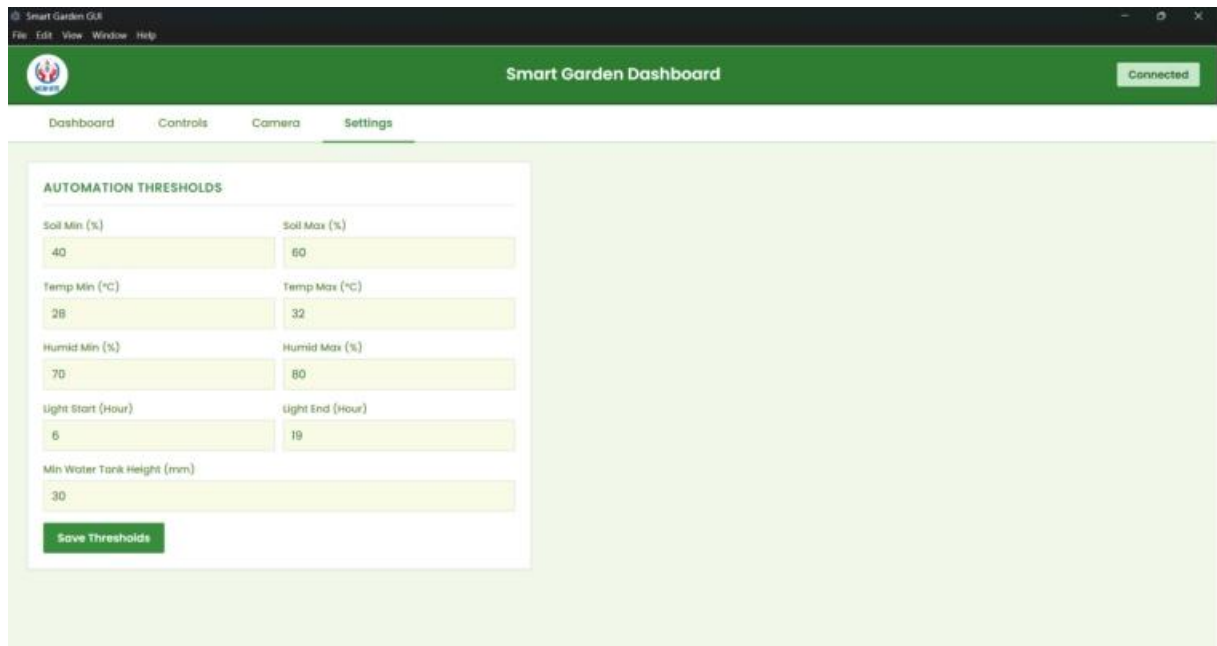
Tương tự như điều khiển ở chế độ thủ công, người dùng tiến hành vào tab “Controls” nhưng lần này thay vì để “Automation Mode” là OFF thì ta đặt đó là ON. Khi đó, server sẽ dựa vào dữ liệu của cảm biến và các ngưỡng được đặt bởi người dùng để gửi tín hiệu bật tắt thiết bị mà không cần sự can thiệp của người dùng.

Trước khi bắt đầu đưa hệ thống hoạt động ở chế độ tự động, trước hết cần phải thiết lập các ngưỡng hoạt động mong muốn để server biết khi nào nên bật hay tắt một thiết bị. Để làm được việc này người dùng cần vào tab “Settings”. Ở đây sẽ có các trường thông tin người dùng có thể điều chỉnh theo ý muốn. Các trường thông tin hoạt động như sau:

- Điều khiển bơm: Cần phải đạt đủ 2 điều kiện để bơm hoạt động là mực nước hiện tại phải trên giá trị “Min Water Tank Height” và độ ẩm đất dưới Soil Min.
- Điều khiển đèn: Thời gian hiện tại cần phải nằm trong khoảng “Light Start” và “Light End”.

- Điều khiển quạt 1: Nhiệt độ hiện tại cần phải trên Temp Max. Khi dưới Temp Min thì quạt 1 sẽ tắt. Khoảng giữa là khoảng giữa trạng thái.
- Điều khiển quạt 2: Độ ẩm không khí hiện tại cần phải trên Humid Max. Khi dưới Humid Min thì quạt 2 sẽ tắt. Khoảng giữa là khoảng giữa trạng thái.

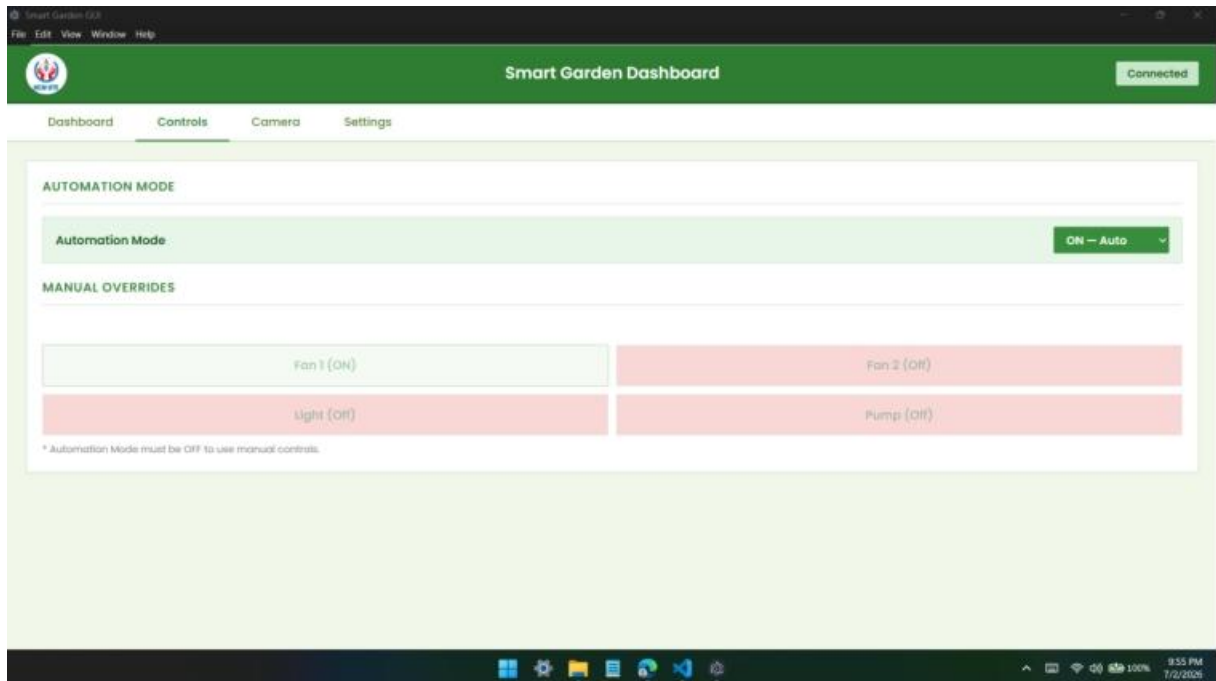
Sau khi đã lựa chọn xong các ngưỡng cho chế độ tự động, nhấn nút “Save Thresholds” để lưu lại các giá trị này cho server. Khi đó, việc điều khiển các thiết bị này sẽ được tự động điều khiển bởi server.



Hình 4.19: Giao diện tab điều chỉnh ngưỡng hoạt động cho chế độ tự động

```
[CMD TASK] Received command id=1299 cmd=3FFCE778
[CMD TASK] Serial2 SENT: REQ CMD
[CMD TASK] STM RESPOND: ACK
[CMD TASK] Serial2 SENT: RLY 1
[CMD TASK] STM RESPOND: EXE OK
```

Hình 4.20: ESP32 nhậ lệnh điều khiển quạt 1



Hình 4.21: Hệ thống tự động bật thiết bị dựa trên thông tin môi trường đã thiết lập ngưỡng

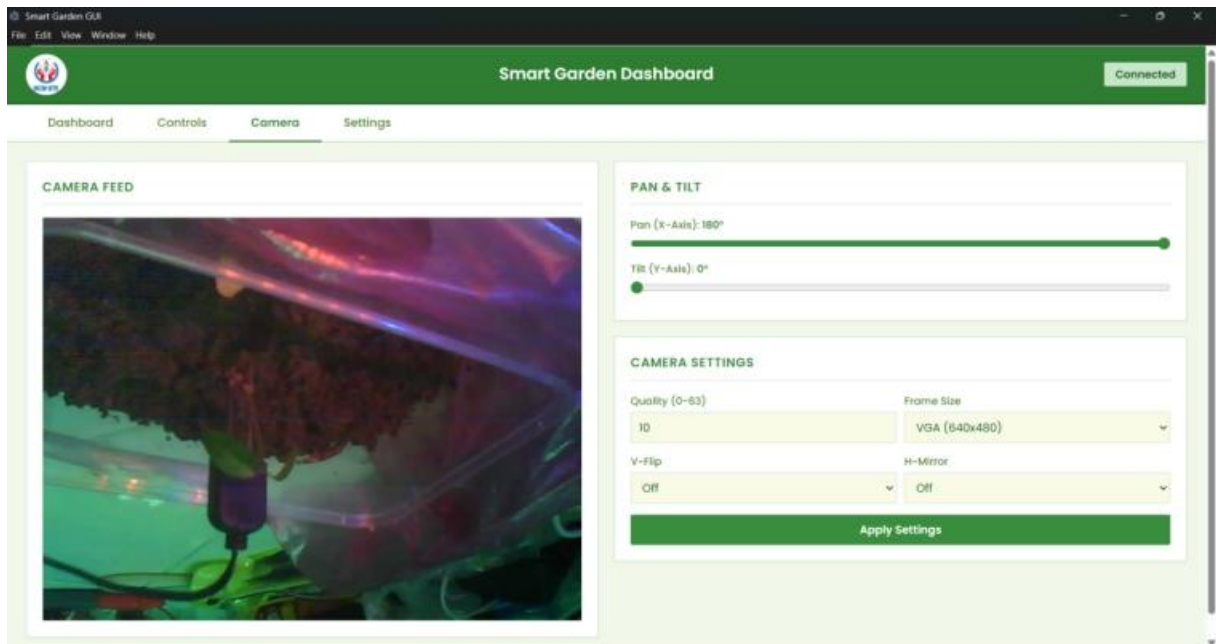
Nếu đặt các ngưỡng như ở hình 4.19, khi nhiệt độ đo được là 33.01, server sẽ tự bật quạt 1 còn các thiết bị còn lại sẽ ở trạng thái OFF vì điều kiện bật không đạt yêu cầu.

Việc thiết lập một khoảng khoảng giữa hai giá trị Min và Max thay vì chỉ dùng một ngưỡng duy nhất để bật tắt quạt, là một kỹ thuật chống dao động thường thấy trong các hệ thống điều khiển tự động. Nếu chỉ sử dụng một ngưỡng duy nhất, khi nhiệt độ môi trường dao động quanh giá trị ngưỡng đó, quạt sẽ liên tục bật rồi tắt trong thời gian ngắn, gây hao mòn relay và tiêu tốn điện năng không cần thiết. Với cơ chế hai ngưỡng, quạt chỉ tắt hẳn khi nhiệt độ đã giảm xuống dưới Temp Min, đảm bảo mỗi chu kỳ bật tắt cách nhau một khoảng đủ lớn, giúp thiết bị hoạt động ổn định và bền hơn.

Đối với điều khiển bơm, việc yêu cầu đồng thời hai điều kiện đóng vai trò như một cơ chế bảo vệ kép vừa đảm bảo cây được tưới khi thực sự cần thiết vừa tránh trường hợp bơm hoạt động khi bể chứa nước đã cạn, có thể dẫn đến bơm chạy khô và hư hỏng động cơ bơm về lâu dài.

2.5. Quan sát cây trồng với camera feed

Từ tab Camera của GUI, người dùng sẽ có một khung hình bên trái là camera feed được gửi trực tiếp từ ESP32 CAM và bên cạnh là các điều khiển servo cho phép di chuyển góc của camera. Bên dưới là các lựa chọn cho phép người dùng điều chỉnh khung hình camera tùy theo ý muốn.

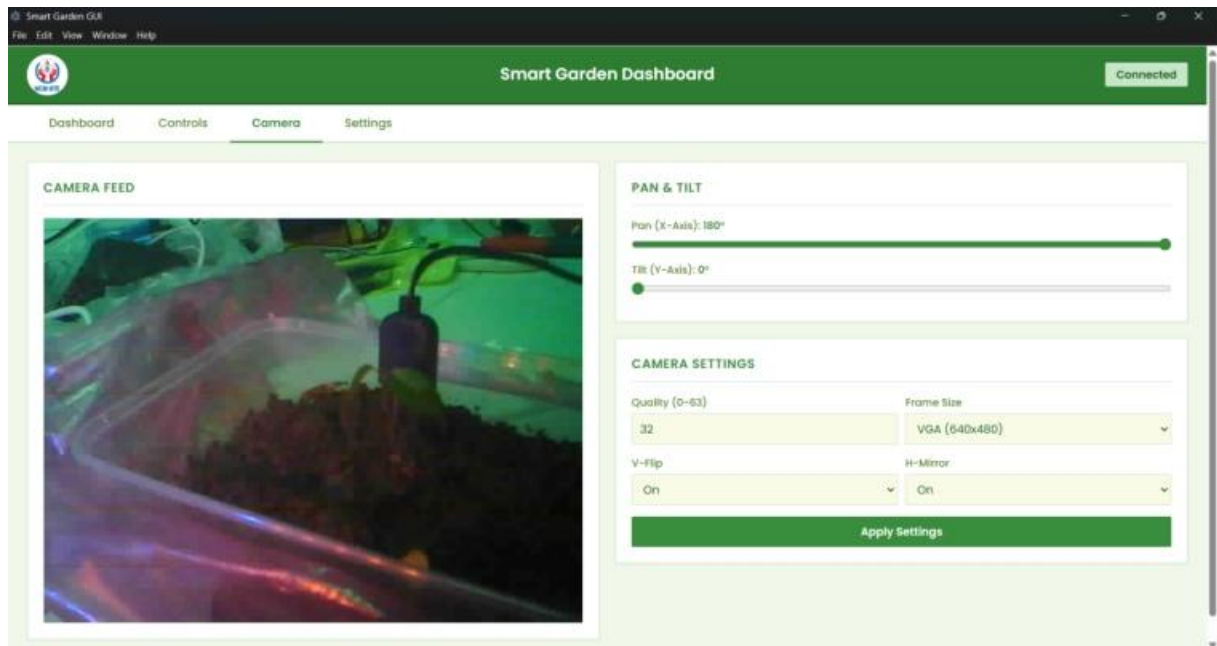


Hình 4.22: Tùy chỉnh các thông số Camera

Để điều chỉnh các frame mà camera gửi, người dùng có thể chọn thay đổi 4 các lựa chọn sau:

- Quality: Mức chất lượng nén JPEG của ảnh (giá trị thấp hơn cho chất lượng cao hơn và kích thước tệp lớn hơn).
- Frame size: Độ phân giải camera.
- V-Flip: Lật camera frame theo chiều dọc.
- H-Mirror: Lật camera frame theo chiều ngang.

Từ cấu hình mặc định như hình 4.22, khi ta chọn các lựa chọn mới ở hình và nhấn nút “Apply Settings”, các cấu hình đó sẽ được áp dụng cho ESP32 CAM và kết quả camera feed được hiển thị trên GUI sẽ thay đổi tương ứng theo lựa chọn đó.



Hình 4.23: Camera thay đổi theo thông số đã chỉnh

Việc cho phép người dùng tùy chỉnh Quality và Frame size ngay trên GUI thay vì cố định một cấu hình duy nhất giúp cân bằng giữa chất lượng hình ảnh quan sát và băng thông mạng tiêu thụ. Trong trường hợp kết nối WiFi yếu hoặc nhiều thiết bị cùng truy cập server, người dùng có thể chủ động giảm Frame size hoặc tăng mức nén JPEG để đảm bảo luồng video vẫn mượt mà, tránh hiện tượng giật, lag hoặc mất kết nối camera feed giữa chừng. Đây điều rất quan trọng khi cần theo dõi tình trạng cây trồng theo thời gian thực.

PHẦN V: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Đề tài đã xây dựng thành công hệ thống quản lý vườn thông minh ứng dụng IoT với kiến trúc 3 nốt (MCU, Server, GUI). Hệ thống có khả năng thu thập các thông số môi trường (nhiệt độ, độ ẩm, ánh sáng, mực nước), điều khiển tự động và thủ công các thiết bị (đèn, quạt, bơm) thông qua giao diện GUI trực quan. Kết nối WebSocket đảm bảo tốc độ phản hồi nhanh, Tailscale VPN cho phép truy cập từ xa an toàn, cơ chế phục hồi và lưu hàng đợi giúp hệ thống vận hành ổn định.

Tuy nhiên, hệ thống mới chỉ thử nghiệm trên quy mô nhỏ, số lượng cảm biến và thiết bị còn hạn chế, chưa có cơ chế cảnh báo thông minh và chưa tối ưu trên đa nền tảng.

2. Hướng phát triển

- Triển khai trên diện tích lớn với nhiều cụm cảm biến và thiết bị phân tán.
- Ứng dụng AI để dự đoán nhu cầu tưới tiêu và phát hiện bất thường.
- Xây dựng app Android/iOS để tiện lợi hơn cho người dùng.
- Cảnh báo thông minh, gửi cảnh báo qua email, SMS khi thông số vượt ngưỡng.

TÀI LIỆU THAM KHẢO

1. Ali, M., Kanwal, N., Hussain, A., Samiullah, F., Iftikhar, A., & Qamar, M. (2020). *IoT based smart garden monitoring system using NodeMCU microcontroller*. Journal of Independent Studies and Research (JISR), 18(1), 1-8. Truy cập:
<https://pdfs.semanticscholar.org/daa8/3ec88c3b15f6a0f9f726b4a3dfb2346469cc.pdf>
2. Bicomumakuba E., Reza M.N., Jin H., Samsuzzaman, Lee K.H., & Chung S.O (2025). *Multi-Sensor Monitoring, Intelligent Control, and Data Processing for Smart Greenhouse Environment Management*. Sensors. MDPI. Truy cập:
<https://doi.org/10.3390/s25196134>
3. Đoàn, H. C., Phạm, Đ. T., & Đỗ, T. H. (2024). *Xây dựng hệ thống giám sát các thông số môi trường nhà vườn thông minh sử dụng IoT*. Tạp chí Khoa học và Công nghệ Trường Đại học Thành Đông, (14), 5-9. Truy cập:
<https://scholar.dlu.edu.vn/thuvienso/bitstream/DLU123456789/281050/1/103329-2101-213304-1-10-20241008.pdf>
4. Maslan, M. N., Ramli, I. R., Md Fauadi, M. H. F., Ayob, M. E., Baharom, M. F., Ghazali, I., & Tanjung, T. (2025). *Design and implementation of outdoor home smart farming based on raspberry pi for home assistant management*. Multidisciplinary Science Journal. Truy cập:
<https://eprints.utm.edu.my/id/eprint/29571/2/0220512022025105211661.pdf>
5. Thamaraimanalan, T., Vivekk, S. P., Satheeshkumar, G., & Saravanan, P. (2018). *Smart garden monitoring system using IOT*. Asian Journal of Applied Science and Technology (AJAST), 2(2), 186-192. Truy cập
<https://ajast.net/data/uploads/4026.pdf>